

Ingeniería y Calidad de Software

Resumen de estudio para el parcial de **Aprobación Directa**, que toma el temario completo: las nueve unidades del programa. Sigue el estilo del resumen de Regularización: está pensado para leerse de corrido, cada capítulo abre explicando por qué el tema existe antes de entrar en las definiciones, y las tablas aparecen sólo donde hay algo que comparar. El orden es temático, no el del examen. Los capítulos 24 y 25 están pensados para consultarlos durante el parcial. Para el detalle completo de cada tema y los ejercicios resueltos está la wiki ([ISW.md](#)).

CONTENIDO

- | | |
|---|--|
| 1 El parcial y su temario | 14 Los requerimientos y su desarrollo |
| 2 Qué significa calidad en software | 15 La gestión de requerimientos y de los cambios |
| 3 El software como objeto de ingeniería | 16 Verificación y validación |
| 4 CMMI: qué es y para qué sirve | 17 La organización de las pruebas |
| 5 Anatomía de un área de proceso | 18 Las técnicas dinámicas |
| 6 Los niveles de madurez | 19 Las técnicas estáticas: revisiones |
| 7 La gestión de procesos de la organización | 20 La gestión de configuración (CM) |
| 8 Activos de proceso y productos de trabajo | 21 El aseguramiento de la calidad (PPQA) |
| 9 El proceso de desarrollo: SPEM y RUP | 22 La medición y el análisis (MA) |
| 10 El proyecto y su planificación | 23 Las pericias informáticas |
| 11 La estimación del esfuerzo | 24 Cómo reconocer el área de proceso en un enunciado |
| 12 La gestión de riesgos (RSKM) | 25 Las preguntas BP |
| 13 Los ciclos de vida | 26 Confusiones frecuentes |

1. El parcial y su temario

El parcial de Aprobación Directa es **presencial**, de **opción múltiple** —entre 15 y 20 preguntas, según la cátedra, aunque la cantidad es orientativa— y **a libro abierto, sólo en papel**: no se permite ningún dispositivo electrónico. Se aprueba con el **60 % del total de puntos**. Para promocionar la materia hay que aprobar los dos parciales, Regularización y Aprobación Directa, y entregar los trabajos prácticos de clase. Si se desaprueba éste, queda el **Globalizador** de Aprobación Directa.

A diferencia del parcial de Regularización, acá no hay recorte de páginas: entra **todo el programa**.

UNIDAD	TEMA	CAPÍTULOS
1 — Modelos de calidad	Calidad, ingeniería del software, CMMI, niveles de madurez	2 a 6
2 — Gestión de procesos	OPE, OPD, OT, activos de proceso, SPEM y RUP	7 a 9
3 — Gestión de proyectos	PP, PMC, estimación y puntos función, riesgos, ciclos de vida	10 a 13
4 — Requerimientos	RD, REQM y gestión de solicitudes de cambio	14 y 15
5 — Verificación y validación	VER, VAL, pruebas, técnicas dinámicas y estáticas	16 a 19
6 — Gestión de configuración	CM: elementos, líneas base, versionado, cambios	20

UNIDAD	TEMA	CAPÍTULOS
7 — Aseguramiento de calidad	PPQA: auditorías, no conformidades, escalamiento	21
8 — Medición y análisis	MA: métricas de proceso, proyecto y producto	22
9 — Pericias informáticas	Evidencia digital, cadena de custodia, protocolo	23

Lo que muestran los parciales de 2024 y 2025. Aunque entra todo, el peso estuvo en las unidades que **no** estaban en el parcial de Regularización: **configuración, aseguramiento de la calidad, requerimientos y medición**, más alguna pregunta de planificación y de verificación y validación. Riesgos y pericias no aparecieron, pero entran.

Las preguntas tienen tres formas que se repiten. La primera es la pregunta **BP**: se describe cómo trabaja una software factory y hay que decir si esa práctica **puede generar problemas de calidad** y por qué; el capítulo 25 explica cómo resolverlas. La segunda pide reconocer **qué área de proceso —y a veces qué práctica— corresponde a una actividad** descripta; para eso está el capítulo 24. La tercera son ejercicios cortos con números: calcular cuántos años sobrevive un esquema de versión, cuántas releases mínimas tiene un proyecto, o qué casos de prueba salen de una partición.

ATENCIÓN En las preguntas de selección múltiple, **marcar una opción incorrecta además de la correcta deja la pregunta en cero**. Pasó en el parcial de 2024. Ante la duda, es mejor marcar sólo lo que se puede justificar.

2. Qué significa calidad en software

La palabra calidad arrastra un problema: todos creen saber qué quiere decir, y cada uno entiende algo distinto. En la materia se trabaja con una idea central —**calidad es idoneidad de uso**— y varias definiciones que la precisan desde ángulos diferentes. En una opción múltiple, cada una se reconoce por la palabra que aporta.

La definición **clásica**, de Juran, la plantea en dos mitades: las características del producto que satisfacen las necesidades del cliente, y la **inexistencia de deficiencias**. Un producto puede tener todas las funciones pedidas y aun así fallar la segunda mitad. **Deming** la mira desde el otro lado: calidad es lo que **aumenta la satisfacción del cliente**.

CMMI la define como la capacidad de un conjunto de características inherentes de un producto, componente o **proceso** de satisfacer **por completo** los requisitos del cliente. Lo importante acá es que incluye al proceso: para CMMI, un proceso también tiene calidad.

ISO 8402 habla del conjunto de propiedades y características que le confieren aptitud para satisfacer necesidades **explícitas o implícitas**. La palabra que aporta es *implícitas*: hay expectativas que el cliente nunca escribió y que igual espera que se cumplan. **ISO 9000** la vuelve una cuestión de grado: el **nivel** al que las características inherentes satisfacen los requisitos.

2.1 De qué depende la calidad

Cuatro elementos la determinan: los **procesos y buenas prácticas** que se siguen, las **herramientas** disponibles, las **personas** que hacen el trabajo, y las **medidas y métricas** con las que se controla. Ninguno alcanza por sí solo. Un equipo excelente con un proceso caótico produce resultados irrepetibles, y un proceso impecable no compensa a un equipo sin formación.

2.2 Tres tipos de calidad de producto

No toda la calidad se ve desde afuera, y esa distinción explica por qué a veces un sistema que "anda bien" es un problema.

La **calidad interna** es la que **el cliente no percibe**: facilidad de mantenimiento, claridad del código, respeto de los estándares, reusabilidad. No la ve, pero la paga: es la que determina cuánto cuesta cada cambio futuro.

La **calidad externa** es la visible: que el producto cumpla los requerimientos acordados.

La **calidad de uso** es la que algunos llaman *calidad futura*. Es la que no se tuvo en cuenta y aparece recién cuando el software se usa de verdad: el sistema pasa todas las pruebas, se entrega, y a la media hora de operación real se nota el problema que nadie había previsto.

2.3 Los tres niveles de gestión de la calidad

La calidad se puede gestionar en tres escalas distintas, y cada una tiene sus herramientas.

A nivel de **producto** se gestiona con pruebas ejecutadas en paralelo a cada etapa del desarrollo. A nivel de **proyecto** se gestiona controlando las fases y las áreas de gestión de ese proyecto en particular. A nivel de **proceso** se gestionan las áreas de proceso de **toda la organización** mediante una metodología.

Ese tercer nivel es el que ocupa la mayor parte de la materia, y es donde juega CMMI. La apuesta de fondo es que **la calidad de un producto está muy influenciada por la calidad del proceso empleado para desarrollarlo y mantenerlo**: si el proceso es bueno, los productos buenos dejan de ser casualidad.

Conviene fijar desde ya la diferencia entre **proceso** y **proyecto**, porque atraviesa toda la materia. El proceso es **genérico y repetitivo**, no tiene fechas y **produce siempre el mismo producto**; el proyecto es **único**, tiene inicio y fin, y trabaja con restricciones de tiempo, dinero y recursos. Un proyecto es, en el fondo, una instancia de los procesos de la organización.

2.4 Aseguramiento y control de la calidad

Dentro de la gestión de la calidad conviven dos actitudes que se confunden seguido: **prevenir** los defectos o **detectarlos**.

	QA – ASEGURAMIENTO	QC – CONTROL
Naturaleza	Preventivo	Reactivo
Mira	El proceso	El producto
Ejemplos	Auditorías de proceso, definición de procesos, formación	Revisiones, inspecciones, pruebas

El ejemplo de clase lo deja claro. Una fábrica produce mil tazas, al final controla que pesen menos de 150 gramos y descarta las diez que no cumplen: eso es **control**. Si salen quinientas defectuosas y, en vez de descartarlas, se revisa el proceso productivo para que no vuelva a pasar, eso es **aseguramiento**. El área PPQA de CMMI (capítulo 21) es aseguramiento en este sentido; las pruebas son control.

ATENCIÓN La presentación de INTECO asocia además la verificación con QA y la validación con QC. Eso **no coincide con CMMI**, donde la prueba de sistema es verificación. Para ubicar una actividad en un área, usá siempre CMMI (capítulo 16).

3. El software como objeto de ingeniería

Antes de hablar de procesos conviene entender por qué el software necesita una ingeniería propia y no le sirven las de otras disciplinas.

Software = programas + datos + documentos. Es un elemento **lógico**, no físico.

La percepción común —y equivocada— es que el software son sólo los programas. La norma IEEE 610 agrega un cuarto componente, los procedimientos, pero en los parciales la respuesta esperada fue la de INTECO: **programas, datos y documentos**.

Esa naturaleza lógica trae cuatro consecuencias que condicionan todo lo demás.

El software se desarrolla, no se fabrica. No hay línea de producción: cada producto se construye para los requisitos únicos de un cliente. No se pueden amortizar costos repitiendo unidades, porque no hay unidades que repetir.

El recurso principal son las personas, y no son intercambiables con el tiempo. Agregar gente no acelera un proyecto de forma lineal: el desarrollo requiere coordinación y comunicación, y cada integrante nuevo no es productivo de inmediato y además consume tiempo de los que ya están. Por eso es falso el principio de que "las personas y el tiempo son intercambiables".

El software no se estropea, pero se deteriora. Un engranaje se desgasta por uso; el software no. Lo que lo degrada son los **cambios**: cada modificación de mantenimiento tiene probabilidad de introducir defectos nuevos, y con los años el producto acumula complejidad y fragilidad.

La reutilización está lejos de su potencial. Identificar componentes reutilizables es difícil justamente porque cada producto se construye para requisitos únicos.

3.1 Qué es la ingeniería del software

Ingeniería del software (IEEE): aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software.

Las tres palabras importan. *Sistemático* excluye improvisar; *disciplinado* excluye abandonar el método cuando aprieta el tiempo; *cuantificable* excluye opinar sin medir.

Se suele representar como una **tecnología multicapa**: sobre una base de **compromiso con la calidad** se apoyan los **procesos**, sobre ellos los **métodos**, y encima las **herramientas**. El orden no es decorativo: comprar herramientas sin proceso debajo no produce calidad. El proceso es la capa que mantiene unidas a las demás: define quién hace qué, cómo se coordinan las actividades y dónde están los **puntos de control de calidad**. Todo equipo tiene un proceso, aunque muchas veces sea ad hoc, invisible y caótico.

3.2 Las etapas y el mantenimiento

Las etapas clásicas son: análisis de requisitos, especificación, diseño y arquitectura, programación, **prueba** y mantenimiento. La etapa que comprueba que el software realiza correctamente las tareas indicadas en la especificación es la de **prueba**, y es buena práctica que pruebe alguien distinto de quien programó.

El mantenimiento no es una sola cosa. Distinguir los cuatro tipos es una de las preguntas recurrentes de la materia:

TIPO	QUÉ PERSIGUE
Correctivo	Corregir errores detectados en el producto
Evolutivo	Incorporaciones, modificaciones y eliminaciones para cubrir la expansión o el cambio en las necesidades del usuario
Adaptativo	Responder a cambios del entorno donde opera el sistema: hardware, software de base, base de datos, comunicaciones
Perfectivo	Mejorar la calidad interna del sistema, sin cambio funcional visible

ATENCIÓN **Perfectivo no es sinónimo de evolutivo:** el evolutivo cambia lo que el sistema **hace**; el perfectivo mejora **cómo está hecho**. Y que un cambio lo pida el cliente no lo vuelve evolutivo: llevar el sistema a sucursales de otra ciudad, que la tecnología actual no soporta, es **adaptativo**, porque lo que cambia es el entorno.

3.3 Los principios de la disciplina

De la lista de principios de INTECO, dos aparecen una y otra vez como correctos: **haz de la calidad la razón de trabajar y probar, probar y probar**. Otros dos que conviene reconocer: el **compromiso del cliente es el factor más crítico** de la calidad, e **itera en todas las fases excepto en la codificación**. Y tres aparecen como distractores porque enuncian exactamente lo contrario de lo que sostiene la disciplina: que las personas y el tiempo son intercambiables, que conviene hacerlo rápido primero y correcto después, y que hay que forzar el mismo modelo de ciclo de vida en todos los proyectos.

4. CMMI: qué es y para qué sirve

CMMI son las siglas de *Capability Maturity Model Integration*: modelo de madurez de capacidades integrado, desarrollado por el SEI. La cátedra usa la versión **CMMI-DEV v1.2** en castellano. No es una metodología ni un manual de procedimientos. Es una **guía de buenas prácticas** que no dice *cómo* hacer las cosas, sino **qué** hay que lograr.

Esa distinción es la clave para entender el modelo entero. CMMI define objetivos; cada organización decide con qué prácticas los alcanza, y puede usar prácticas propias distintas de las que el modelo sugiere, siempre que cumpla el objetivo.

Trabajar con un modelo probado da tres cosas que una organización sola tarda años en construir: la **experiencia acumulada** de otras empresas, un **lenguaje y una visión común** dentro de la organización, y un **punto de partida** para no inventar desde cero. Los resultados que se le atribuyen son menos defectos, menos tiempo de entrega, menor costo, más satisfacción del cliente y más beneficios.

Hay algo que CMMI **no** hace: agregar actividades técnicas. Una organización sin ningún nivel también analiza, diseña, programa y prueba. Lo que cambia con la madurez es si esas actividades están **definidas, institucionalizadas, medidas y mejoradas**.

4.1 Área de proceso

Área de proceso: grupo de prácticas relacionadas que, implementadas conjuntamente, satisfacen un conjunto de objetivos importantes para la mejora en esa área.

CMMI define **22 áreas de proceso**, repartidas en los niveles de madurez 2 a 5 y agrupadas en cuatro categorías: **gestión de procesos**, **gestión de proyectos**, **ingeniería** y **soporte**. Cada una cubre un aspecto del trabajo: planificar proyectos, gestionar requerimientos, verificar productos, formar gente.

5. Anatomía de un área de proceso

Toda área de proceso está construida con las mismas piezas, y esas piezas tienen distinto peso. Entender cuál es obligatoria y cuál no es probablemente el concepto más preguntado de la materia.

CATEGORÍA	QUÉ ES	QUÉ INCLUYE
Requeridos	Lo que la organización debe lograr para satisfacer el área. Es la base de las evaluaciones	Metas específicas (SG) y metas genéricas (GG)
Esperados	Lo que puede implementar para lograr lo requerido. Se admiten alternativas propias	Prácticas específicas (SP) y prácticas genéricas (GP)
Informativos	Material que ayuda a entender cómo aproximarse a lo requerido y lo esperado	Subprácticas, productos de trabajo típicos, ampliaciones, elaboraciones de prácticas genéricas, notas, ejemplos, referencias, declaración de propósito, áreas relacionadas

ATENCIÓN Lo **requerido son sólo las metas**. Las prácticas —aunque el modelo las liste y las explique— son **esperadas**, no obligatorias. Una organización puede sustituir una práctica por otra si con ella alcanza la misma meta. Las subprácticas son informativas, aunque en los ejercicios de discriminación son las que justifican la respuesta (capítulo 24).

5.1 Qué significa "genérico"

Un componente es **genérico** cuando la misma declaración se aplica a **múltiples áreas de proceso**. Las metas y prácticas específicas son propias de un área; las genéricas se repiten en todas.

El papel de las genéricas es tratar la **institucionalización**: que el proceso no dependa de la buena voluntad de quien lo ejecuta, sino que esté incorporado a la manera de trabajar de la organización. Esa es la respuesta cuando preguntan qué componentes tratan la institucionalización.

Dos genéricas aparecen en los ejercicios. La **GP 2.9 Evaluar objetivamente la adherencia** está en todas las áreas y se implementa a través de PPQA (capítulo 21). La **GP 2.6 Gestionar configuraciones** pone bajo control los productos de cada área con el sistema de gestión de configuración (capítulo 20).

5.2 La numeración

Las metas se numeran secuencialmente: **SG1**, **SG2**, **GG1**. Las prácticas llevan dos números, **SP x.y**, donde **x es el número de la meta** a la que pertenecen e **y el número de secuencia** dentro de esa meta. Así, **SP 2.3** es la tercera práctica de la segunda meta específica.

Saber leer la numeración sirve para descartar opciones: si un área tiene dos metas específicas, una opción que diga SP 3.1 es imposible.

6. Los niveles de madurez

CMMI organiza la mejora en cinco escalones. Cada uno describe un estado de la organización, no una lista de tareas cumplidas.

NIVEL	NOMBRE	CÓMO SE RECONOCE
1	Inicial	Caos: el éxito depende de la heroicidad del personal. Suelen entregar productos que funcionan, pero exceden el presupuesto y el calendario, abandonan los procesos en las crisis y no pueden repetir sus éxitos
2	Gestionado	Los proyectos se planifican y se monitorizan , y la disciplina se mantiene bajo estrés . Los procedimientos pueden ser distintos en cada proyecto
3	Definido	Existe un conjunto de procesos estándar de la organización y cada proyecto adapta el suyo desde ahí, siguiendo guías de adaptación . El rendimiento es predecible, pero sólo cualitativamente
4	Gestionado cuantitativamente	Se gestiona con objetivos cuantitativos y estadística . Se tratan las causas especiales de variación. El rendimiento pasa a ser predecible cuantitativamente
5	En optimización	Mejora continua . Se atacan las causas comunes de variación y se cambia el proceso para mejorar su rendimiento

Causa especial: la propia de una **circunstancia transitoria**, que no forma parte del proceso. **Causa común:** la variación que producen las **interacciones normales** entre los componentes del proceso.

Un servidor que llega diez días tarde y atrasa un proyecto es una causa especial. Una técnica de estimación que subestima siempre los requerimientos complejos es una causa común: está metida en el proceso, y se va a repetir en cada proyecto hasta que se cambie la técnica.

6.1 Las dos comparaciones que hay que poder explicar

Nivel 2 frente a nivel 3: el alcance de los estándares. En nivel 2 cada proyecto puede tener sus propios procedimientos; lo que se exige es que los tenga, los siga y los sostenga. En nivel 3 los procedimientos **se adaptan desde el conjunto estándar de la organización**, y por lo tanto son consistentes entre proyectos salvo por las diferencias que las guías de adaptación permitan.

Nivel 4 frente a nivel 5: el tipo de variación que se trata. El nivel 4 ataca las **causas especiales** —las anomalías puntuales— y logra predictibilidad estadística. El nivel 5 ataca las **causas comunes**, las que están incorporadas al proceso mismo, y para eliminarlas **cambia el proceso**.

6.2 Cómo se avanza: las reglas del modelo escalonado

De estas tres reglas salen casi todos los ejercicios de niveles.

Primera: para estar en un nivel hay que satisfacer todas las áreas de ese nivel y de los anteriores. Si falta una sola área, no se está en ese nivel. Una organización que cumple perfectamente las áreas de nivel 3 pero incumple una de nivel 2 no está en nivel 3 ni en nivel 2: está en **nivel 1**, que es el nivel por defecto porque no exige nada. Si lo que falta es de nivel 3, queda **como máximo en nivel 2**, y sólo si cumple todas las de nivel 2.

Segunda: los niveles son acumulativos. Alcanzar un nivel superior no exime de las metas de los anteriores. Una organización nivel 5 sigue planificando y monitorizando proyectos todos los días. Si dejara de hacerlo, se caería hasta nivel 1.

Tercera: se puede instanciar un área de nivel superior al propio si el contexto de negocio lo justifica, pero se corre el riesgo de intentar prácticas **sin la base institucional que las soporte**. Eso funciona hasta que aparece el estrés, que es justamente cuando más se las necesita. Por eso saltar niveles es, en general, contraproducente.

ATENCIÓN "El aseguramiento de la calidad nace en el nivel 3" es **falso**: PPQA es de nivel 2. Lo que no se puede dar por supuesto en una organización de nivel 2 son los estándares organizacionales, que son de OPD, nivel 3.

6.3 Madurez y capacidad

CMMI admite dos formas de mirar el progreso.

La **madurez**, que es la representación **por etapas**, mide a la **organización entera** contra los conjuntos fijos de áreas de cada nivel. Es la que se certifica y la que usan todos los ejercicios de la materia.

La **capacidad**, que es la representación **continua**, mide el nivel alcanzado en **un área de proceso puntual**, con independencia del resto. Sirve para que la organización elija en qué área quiere crecer primero, según lo que le duela al negocio. En la versión 1.2 los niveles de capacidad van de **0 a 5**; los resúmenes que dicen 0 a 3 hablan de la versión 1.3.

6.4 Qué áreas hay en cada nivel

NIVEL ÁREAS DE PROCESO

2	PP Planificación de proyecto · PMC Monitorización y control del proyecto · REQM Gestión de requerimientos · MA Medición y análisis · PPQA Aseguramiento de la calidad de proceso y de producto · CM Gestión de configuración · SAM Gestión de acuerdos con proveedores
3	OPF Enfoque en procesos · OPD Definición de procesos · OT Formación organizativa · VER Verificación · VAL Validación · RD Desarrollo de requerimientos · RSKM Gestión de riesgos · TS Solución técnica · PI Integración de producto · IPM Gestión integrada de proyecto · DAR Análisis de decisiones y resolución
4	OPP Rendimiento de procesos de la organización · QPM Gestión cuantitativa de proyecto
5	OID Innovación y despliegue en la organización · CAR Análisis causal y resolución

En negrita, las trece que este resumen desarrolla; del resto alcanza con el nivel. Las categorías: **gestión de procesos** son OPE, OPD, OT, OPP y OID; **gestión de proyectos**, PP, PMC, SAM, IPM, RSKM y QPM; **ingeniería**, REQM, RD, TS, PI, VER y VAL; y **soporte**, CM, PPQA, MA, DAR y CAR.

ATENCIÓN La traducción castellana tiene erratas: en las carátulas, **OT** figura como nivel 4, **OID** como nivel 3 y **RSKM** como nivel 2. Lo correcto, según las tablas del mismo libro, es **OT = 3**, **OID = 5** y **RSKM = 3**.

6.5 Nivel 1 y nivel 5 frente al mismo proyecto

Un ejercicio de clase muestra qué cambia de verdad entre niveles. Ante el mismo imprevisto —el servidor llega diez días tarde y el cliente pide cambiar una regla de negocio—, la organización de **nivel 1** estima a ojo, hace horas extra, se saltea las pruebas y acuerda el cambio por teléfono: **abandona el proceso en la crisis**. La de **nivel 5** ya tenía el retraso registrado como riesgo, hace entrar el cambio por gestión de requerimientos y de confi-

guración, y al cierre busca las causas comunes de los desvíos: **el proceso no se abandona, se ejecuta**. Lo que se olvida son las similitudes: las dos **pueden entregar software que funciona**, hacen las mismas actividades técnicas y las dos **pueden fallar**; el nivel 5 no garantiza cero defectos.

7. La gestión de procesos de la organización

Tres áreas de nivel 3 se ocupan de los procesos de la organización como un todo, y son las que más se confunden entre sí porque sus nombres se parecen. La forma más rápida de separarlas es por el verbo que las define.

OPF diagnostica y despliega. OPD define y guarda. OT capacita.

Las tres trabajan **para toda la organización**, no para un proyecto: cuando un enunciado plantea un problema de un proyecto puntual, difícilmente la respuesta sea una de ellas. Lo que hay que dominar de cada una es el **propósito** y sus **metas y prácticas específicas**, que se listan a continuación con el texto oficial.

7.1 OPF – Enfoque en procesos de la organización

Propósito: planificar, implementar y desplegar las mejoras de procesos de la organización, basadas en una comprensión completa de las **fortalezas y debilidades actuales** de los procesos y de los activos de proceso.

Es el área del **diagnóstico y la mejora**. Evalúa dónde está parada la organización —incluso comparándose con otras organizaciones—, arma los planes de acción para corregir las debilidades que encuentra, y después despliega los cambios e incorpora lo aprendido. Sus tres metas siguen exactamente ese recorrido:

- **SG 1 Determinar las oportunidades de mejora de procesos.** SP 1.1 Establecer las necesidades de procesos de la organización · SP 1.2 **Evaluar los procesos** de la organización · SP 1.3 Identificar las mejoras de procesos.
- **SG 2 Planificar e implementar las mejoras de procesos.** SP 2.1 **Establecer planes de acción de procesos** · SP 2.2 Implementar los planes de acción de procesos.
- **SG 3 Desplegar los activos de proceso e incorporar las lecciones aprendidas.** SP 3.1 Desplegar los activos de proceso · SP 3.2 Desplegar los procesos estándar · SP 3.3 Monitorizar la implementación · SP 3.4 **Incorporar las experiencias relativas al proceso en los activos de proceso** de la organización.

ATENCIÓN **SG 2 frente a SG 3** se pregunta siempre. La SG 2 trabaja con mejoras **todavía no aprobadas**, que se prueban en **proyectos piloto**. La SG 3 **despliega** a toda la organización una mejora **ya aceptada**. Fijate si el enunciado habla de una **propuesta** o de una **modificación ya introducida**.

Su lugar entre PPQA y OPD. PPQA **evalúa e informa**: emite no conformidades y tendencias de calidad. OPF **analiza** esos informes y propone la mejora: si varios informes dicen que los proyectos completan mal la plantilla de casos de prueba, OPF puede proponer una guía de llenado. OPD **deja asentada** la mejora en el activo. El sector de SQA de una empresa suele hacer las dos primeras cosas a la vez, así que no hay que confundir el departamento con el área de proceso.

7.2 OPD – Definición de procesos de la organización

Propósito: establecer y mantener un conjunto **usable** de **activos de proceso** de la organización y de **estándares del entorno de trabajo**.

Es el área que **define y guarda**. Todo lo que la organización tiene para que sus proyectos lo reutilicen —el proceso estándar, los modelos de ciclo de vida, las reglas para adaptarlos, el repositorio de mediciones, la biblioteca de documentos— lo establece OPD. Tiene una sola meta que entra en la materia, y cada práctica crea uno de esos activos:

- **SG 1 Establecer los activos de proceso de la organización.** SP 1.1 Establecer los **procesos estándar** · SP 1.2 Establecer las descripciones de los **modelos de ciclo de vida** · SP 1.3 Establecer los **criterios y las guías de adaptación** · SP 1.4 Establecer el **repositorio de medición** de la organización · SP 1.5 Establecer la **biblioteca de activos de proceso** · SP 1.6 Establecer los **estándares del entorno de trabajo**.

La segunda meta, *facilitar la gestión IPPD*, pertenece a la extensión IPPD y no entra.

Tres prácticas se preguntan con un rasgo que las delata. Las **guías de adaptación** (SP 1.3) dicen cómo se arma el proceso de un proyecto a partir del estándar, y son la práctica de las **excepciones**: el procedimiento para pedir que un proyecto no incluya cierta actividad es OPD SP 1.3. El **repositorio de medición** (SP 1.4) guarda **medidas** de los proyectos —"la base de datos donde se registrarán métricas de todos los proyectos"—, mientras que la **biblioteca de activos** (SP 1.5) guarda **documentos**: políticas, plantillas, checklists, lecciones aprendidas —"el sitio de Intranet donde se consultan los documentos del proceso"—.

ATENCIÓN Cuando un enunciado menciona un **repositorio de medidas ligado a los procesos estándar de la organización**, la respuesta es **OPD** (SP 1.4), no MA. MA es de nivel 2 y mide **el proyecto**; el repositorio de la organización es un **activo**, y los activos son de OPD.

ATENCIÓN No toda revisión entre pares es VER. Si tres jefes de proyecto revisan un **procedimiento de la organización aún no publicado**, la respuesta es **OPD SP 1.1**: se revisa un activo, no un producto de un proyecto.

7.3 OT – Formación organizativa

Propósito: desarrollar las **habilidades y el conocimiento** de las personas para que puedan realizar sus **roles** eficaz y eficientemente.

Primero se construye la capacidad de formar y después se forma. Un punto que suele preguntarse es el reparto de responsabilidades: la organización cubre las necesidades de formación **comunes** a los proyectos, y cada proyecto cubre las **específicas** suyas. Las necesidades **estratégicas** miran varios años hacia adelante; el **plan táctico** baja eso a cursos concretos.

- **SG 1 Establecer una capacidad de formación organizativa.** SP 1.1 Establecer las necesidades de formación **estratégicas** · SP 1.2 Determinar qué necesidades de formación son **responsabilidad de la organización** · SP 1.3 Establecer un **plan táctico** de formación · SP 1.4 Establecer la capacidad de formación.
- **SG 2 Proporcionar la formación necesaria.** SP 2.1 Impartir la formación · SP 2.2 Establecer los **registros** de formación · SP 2.3 Evaluar la **eficacia** de la formación.

ATENCIÓN OT no es PP SP 2.5. Elegir qué cursos tienen que hacer los programadores **del proyecto X** antes de empezar es **PP SP 2.5** Planificar el conocimiento y las habilidades necesarios. Conseguir un instructor, o mandar a alguien a formarse para que después capacite a otros, sí es **OT** (SP 1.4), aunque lo aprovechen los proyectos.

7.4 Las prácticas que se cruzan entre OPF y OPD

En los exámenes aparece la misma lista de prácticas preguntada dos veces, una pidiendo las de OPF y otra las de OPD. La regla para no mezclarlas: si la práctica **crea** un activo, es OPD; si **evalúa, planifica, despliega o actualiza** lo que ya existe, es OPF.

PRÁCTICA	ÁREA
Establecer el repositorio de medición de la organización	OPD
Establecer las descripciones de los modelos de ciclo de vida	OPD
Establecer los criterios y las guías de adaptación	OPD
Establecer la biblioteca de activos de proceso	OPD
Evaluar los procesos de la organización	OPF
Establecer planes de acción de procesos	OPF
Desplegar los activos de proceso	OPF
Incorporar las experiencias relativas al proceso en los activos de proceso	OPF

8. Activos de proceso y productos de trabajo

Esta distinción es corta de explicar y sorprendentemente fácil de errar, porque los dos conceptos se refieren a documentos que a veces tienen nombres casi idénticos.

Un **activo** es un artefacto que la organización produce **para ser usado en las tareas de los proyectos**. Un **producto de trabajo** es el resultado concreto de una tarea **de un proyecto**.

La pregunta que resuelve cualquier caso es: *¿esto existe para ser usado en muchos proyectos, o salió de uno?*

Las señales de **activo** son las palabras plantilla, guía, directriz, modelo, estándar, política y checklist. También son activos las **herramientas que la organización adquiere** en lugar de desarrollar, junto con su documentación: el manual de usuario del compilador Java es un activo, porque no es producto de ningún proyecto.

Las señales de **producto de trabajo** son las referencias a un proyecto concreto: "del cliente", "del sistema desarrollado". Una minuta, el código fuente, una regla de negocio, la descripción del proceso de negocio del cliente.

El par que fija el criterio es este: la **plantilla de manual de instalación** es un activo —es el molde, y pertenece a la organización—, mientras que el **manual de instalación del sistema desarrollado para el cliente** es un producto de trabajo —es lo moldeado, y pertenece al proyecto.

Lo mismo vale para las áreas que los tocan. El **caso de uso** es un producto de trabajo: lo verifica **VER**, lo valida **VAL** con el cliente, y **PPQA** controla que respete el estándar. La **plantilla de caso de uso** es un activo: la define **OPD** y la despliega **OPF**.

ATENCIÓN Pregunta trampa: "¿qué área se encarga de verificar y almacenar los **productos de trabajo** a nivel organizacional?" La respuesta es **NINGUNA**. "Nivel organizacional" empuja hacia OPD, pero los productos de trabajo salen de cada proyecto; si la pregunta dijera **activos**, sí sería OPD.

9. El proceso de desarrollo: SPEM y RUP

Los capítulos anteriores hablaron de procesos en abstracto: que hay que definirlos, guardarlos y mejorarlos. Este capítulo baja a cómo se **escribe** un proceso concreto: con qué elementos se describe, quién hace qué, y qué material de apoyo lo acompaña.

9.1 Por qué gestionar procesos

Las organizaciones concentran su mejora en tres **dimensiones críticas**: las **personas**, los **métodos y procedimientos**, y las **herramientas y el equipamiento**. Los procesos son lo que sostiene a las tres. Las áreas de **gestión de procesos** de CMMI contienen las actividades **transversales a los proyectos**, y se dividen en **básicas** —OPF, OPD y OT, las del capítulo 7— y **avanzadas** —OPP y OID—.

9.2 SPEM

SPEM (*Software Process Engineering Meta-Model*): estándar de la OMG —el mismo consorcio que mantiene UML— que establece los elementos clave para representar métodos, ciclos de vida, técnicas, roles, actividades, procesos, metodologías y plantillas de la ingeniería del software.

Es un **meta-modelo**: no describe un proceso en particular, sino el vocabulario con el que se describe cualquier proceso. Por eso su alcance se limita a los **elementos mínimos** necesarios, sin características de un dominio o disciplina en particular, y sirve para procesos de distintos estilos, culturas, niveles de formalismo y ciclos de vida. No es un lenguaje de modelado de procesos en general —una afirmación que aparece como falsa—: está orientado al software.

Lo que aporta es facilitar la **comprensión y la comunicación** entre personas, facilitar la **reutilización**, dar soporte a la **mejora** y a la **gestión** de procesos, y guiar su **automatización**.

La idea central es que todo proceso se representa respondiendo tres preguntas:

PREGUNTA	ELEMENTO	QUÉ REPRESENTA
¿Quién?	Rol	Quién hace el trabajo
¿Qué?	Producto de trabajo	Las entradas que usan las tareas y las salidas que producen
¿Cómo?	Tarea	El esfuerzo a realizar

Además, SPEM define cuatro **niveles de detalle** para representar ese esfuerzo, de mayor a menor. El **delivery process** es un proceso completo, tan complejo como se necesite, que sirve de base para un tipo de proyecto. El **capability pattern** es un fragmento de proceso **reutilizable** más de una vez dentro de un delivery process. La **actividad** es el elemento central, que organiza roles, productos de trabajo y tareas. Y la **tarea** es la porción **más pequeña** de trabajo del modelo.

Un ejemplo recorre los cuatro niveles: el ciclo de vida típico de RUP contiene la disciplina *Entorno*, que contiene la actividad *Preparar el entorno para el proyecto*, que contiene la tarea *Personalizar el proceso de desarrollo para*

el proyecto.

9.3 Los elementos del proceso en RUP

RUP (*Rational Unified Process*) es un proceso **iterativo e incremental**, adaptable y no prescriptivo, construido sobre SPEM. Sus elementos son los siguientes.

Fase. El ciclo de vida se descompone en fases, y cada fase es un **período de tiempo entre dos objetivos importantes**. RUP tiene cuatro. **Concepción** (o Inicial) fija el alcance y los límites del producto y estima el costo global. **Elaboración** estabiliza la arquitectura, los requisitos y los planes, y trata los riesgos arquitectónicos. **Construcción** completa el desarrollo y la prueba de toda la funcionalidad, con versiones alfa y beta. **Transición** entrega el producto: prueba beta con operación en paralelo, conversión de datos y formación de los usuarios.

Disciplina. Una **categorización de tareas** según la similitud de sus preocupaciones y la cooperación del esfuerzo. RUP tiene nueve: modelado de negocio, requisitos, análisis y diseño, implementación, prueba, despliegue, configuración y gestión de cambios, gestión de proyectos, y entorno.

Las fases y las disciplinas son dos ejes distintos: las fases **ordenan el tiempo**, las disciplinas **agrupan el tipo de trabajo**. Por eso se cruzan, y a lo largo de las fases se trabaja en varias disciplinas a la vez. Y aunque las cuatro fases hagan parecer a RUP una cascada, dentro de cada fase hay **iteraciones**, y cada una termina en una versión ejecutable.

Actividad. Agrupa lógicamente elementos de proceso relacionados. **Una disciplina tiene una o más actividades, y una actividad tiene una o más tareas.**

Tarea. Describe una **unidad de trabajo**. La llevan a cabo **roles específicos**, dura entre **unas horas y unos días**, suele afectar a uno o pocos productos de trabajo y puede desglosarse en **pasos**. Una tarea bien descrita reúne el **rol responsable**, los **productos de trabajo de entrada y de salida**, y las guías que la apoyan: **directrices, plantillas y listas de comprobación**.

Ejemplo: la tarea *Desarrollar la visión* la ejecuta el **analista de sistemas**; toma como entrada las **solicitudes del interesado** y produce como salida la **visión**. La apoyan la directriz *Entrevista*, la plantilla *Visión* y la lista de comprobación *Visión*.

Rol. Un conjunto de **habilidades, competencias y responsabilidades** relacionadas: analista de sistemas, arquitecto de software, diseñador, revisor técnico.

Producto de trabajo. Un **resultado significativo del proceso**: los roles los usan para realizar tareas y los producen al realizarlas. Hay tres tipos:

TIPO	QUÉ ES
Artefacto	Un producto tangible , no trivial: un documento, un modelo, el código
Resultado	Un producto intangible : un estado o una consecuencia del trabajo
Entregable	Un empaquetado de otros productos de trabajo, que se entrega a una parte interna o externa

9.4 Las guías

Una **guía** es todo contenido cuyo objetivo principal es **explicar otros elementos** del proceso. La presentación de la cátedra desarrolla ocho tipos:

GUÍA	QUÉ ES
Plantilla	La estructura de un producto de trabajo: secciones, formato y cómo completarlas
Directriz	Indicaciones sobre cómo hacer algo concreto; se aplica a tareas y productos de trabajo
Lista de comprobación	Elementos que deben completarse o verificarse ; se usa en revisiones e inspecciones
Ejemplo	Una instancia parcialmente completa de un producto, para mostrar cómo queda
Concepto	Una idea fundamental , más general que una directriz
Guía de herramientas	Cómo usar una herramienta específica para producir parte de un producto
Documentación	Documentos publicados fuera de RUP a los que el proceso hace referencia
Informe	Un resultado generado automáticamente por una herramienta a partir de otros productos

Las guías se conectan con el capítulo anterior: una plantilla, una directriz o una lista de comprobación de la organización son **activos**, y el documento que se produce al usarlas en un proyecto es un **producto de trabajo**.

10. El proyecto y su planificación

Todo proyecto de software arranca con una promesa: un producto, para una fecha, por un costo. Cumplirla es difícil porque el terreno se mueve —el cliente pide algo nuevo, alguien se enferma, una estimación resulta optimista—, y la gestión de proyectos existe para que esos imprevistos se manejen con método y no a pulmón. CMMI le dedica dos áreas de **nivel 2** que son las dos caras de la misma tarea: **PP** arma el plan y **PMC** lo sigue y lo corrige.

10.1 Qué es un proyecto

Proyecto: conjunto de actividades coordinadas y controladas, con **inicio y fin definidos**, que crea un producto o servicio **único** conforme a requisitos específicos, dentro de límites de tiempo, coste y recursos. Se desarrolla **en pasos**, en lo que se llama elaboración gradual.

Tres precisiones que se derivan de la definición. Un proyecto **siempre tiene fin**: termina cuando se logran los objetivos o cuando se cancela; lo que no termina no es un proyecto sino una operación. Los proyectos de software **no son sólo de desarrollo**: existen los de **mantenimiento** —con los cuatro tipos del capítulo 3— y los de **despliegue o implantación**, y la categoría cambia qué tareas entran en la planificación. Y tener un diagrama de Gantt **no es tener un plan de proyecto**: el Gantt es una vista del cronograma, mientras que el plan incluye alcance, estimaciones, recursos, riesgos, calidad y comunicación.

La contracara del proyecto es el **proceso**, que es **repetitivo y reiterativo y produce siempre el mismo producto**. Esa repetibilidad es exactamente lo que el proyecto no tiene. Entre los participantes conviene distinguir al **patrocinador**, que aporta los recursos financieros, del **cliente o usuario**, que es quien va a usar el producto.

10.2 PP – Planificación de proyecto

Su propósito es **establecer y mantener los planes** que definen las actividades del proyecto: desarrollar el plan, interactuar con las partes interesadas, **obtener el compromiso** con él y mantenerlo. La planificación arranca **con los requerimientos**, que son los que definen producto y proyecto. Y **el plan necesitará corregirse**: cambian los requerimientos y los compromisos, y las estimaciones resultan inexactas. Eso no es un fracaso de la planificación; PP cubre también la replanificación.

- **SG 1 Establecer estimaciones.** SP 1.1 Estimar el **alcance** del proyecto (la EDT) · SP 1.2 Establecer las estimaciones de los **atributos** de los productos de trabajo y de las tareas (tamaño y complejidad) · SP 1.3 Definir el **ciclo de vida** del proyecto · SP 1.4 Determinar las estimaciones de **esfuerzo y de coste**.
- **SG 2 Desarrollar un plan de proyecto.** SP 2.1 Establecer el **presupuesto y el calendario** · SP 2.2 Identificar los **riesgos** del proyecto · SP 2.3 Planificar la gestión de los **datos** · SP 2.4 Planificar los **recursos** del proyecto · SP 2.5 Planificar el **conocimiento y las habilidades** necesarios · SP 2.6 Planificar la **involucración de las partes interesadas** · SP 2.7 Establecer el plan de proyecto.
- **SG 3 Obtener el compromiso con el plan.** SP 3.1 Revisar los planes que afectan al proyecto · SP 3.2 Reconciliar los niveles de trabajo y de recursos · SP 3.3 Obtener el compromiso con el plan.

ATENCIÓN La planificación **incluye** estimar las tareas, determinar los recursos e identificar los riesgos. **No incluye** el pago a los recursos ni la especificación detallada de la arquitectura del software, que corresponde a **TS** (Solución técnica). Y fíjate el **alcance** del enunciado: la formación o el registro de datos planificados **para un proyecto puntual** son PP (SP 2.5 o SP 2.3), no OT ni MA, que trabajan para la organización.

La SP 2.6 decidió una pregunta BP del AD de 2025. La plantilla de plan de proyecto de una software factory pedía incluir **sólo los roles de la software factory**, más responsabilidades de OT, OPF y OPD. **Sí** podía generar problemas de calidad, porque los miembros del proyecto por parte del cliente no sabrían qué tienen que hacer: el plan debe tener los roles de **todas** las partes interesadas relevantes, y el cliente lo es. Lo de OT, OPF y OPD era un segundo error: son áreas de la organización, no roles de un proyecto.

10.3 PMC – Monitorización y control del proyecto

También es de **nivel 2**, y su propósito es proporcionar una comprensión del progreso del proyecto para poder tomar **acciones correctivas apropiadas** cuando el rendimiento se desvía significativamente del plan.

El progreso se mide comparando la calidad de los productos de trabajo, el esfuerzo, el coste y el calendario **reales contra el plan**, en los **hitos o niveles de control** definidos en la EDT. Si hay desvío, las acciones correctivas posibles son la **replanificación**, el establecimiento de **nuevos acuerdos**, o la inclusión de **actividades adicionales de mitigación** dentro del plan actual.

Desvío significativo: aquel que, si se deja sin resolver, **impide al proyecto cumplir sus objetivos**. Por eso es falso que cualquier desvío deba resolverse: el modelo pide criterio, no reacción automática.

- **SG 1 Monitorizar el proyecto frente al plan.** SP 1.1 Monitorizar los **parámetros de planificación** · SP 1.2 Monitorizar los **compromisos** · SP 1.3 Monitorizar los **riesgos** · SP 1.4 Monitorizar la **gestión de los datos** · SP 1.5 Monitorizar la **involucración de las partes interesadas** · SP 1.6 Llevar a cabo **revisiones de progreso** · SP 1.7 Llevar a cabo **revisiones de hitos**.
- **SG 2 Gestionar las acciones correctivas hasta su cierre.** SP 2.1 **Analizar** los problemas · SP 2.2 **Llevar a cabo** las acciones correctivas · SP 2.3 **Gestionar** las acciones correctivas.

Dentro de la segunda meta, el tiempo verbal decide la práctica: "analizando la situación se concluyó que la causa es..." es **SP 2.1**; "se determinó reemplazar al usuario clave", **SP 2.2**; "el cambio se hizo hace dos semanas y ahora se evalúa si resolvió el problema", **SP 2.3**.

Los **puntos de control y monitoreo se definen durante la planificación**, no durante la ejecución. En ejecución se usan.

ATENCIÓN Para distinguir PP de PMC en un enunciado, fíjate dónde está la reunión respecto del momento narrado. Si la reunión está **en el futuro** —"para informarlo en la reunión donde se juntará por primera vez todo el equipo"— todavía se está planificando: es **PP**. Si la reunión **ya ocurrió** —"en la última reunión de avance..."— se está monitorizando: es **PMC**.

10.4 Los diez grupos de procesos de gestión

La guía práctica de gestión de proyectos de INTECO organiza el trabajo en diez grupos. Son buenas prácticas, pero no todos tienen que estar presentes en cada proyecto. Vale la pena tener presente a cuál pertenece cada proceso, porque los ejercicios juegan con procesos que suenan a un grupo y pertenecen a otro.

GRUPO	PROCESOS
Coordinación	Iniciar el proyecto · Desarrollar el plan · Gestionar la ejecución · Supervisar el trabajo · Control integrado de cambios · Cerrar el proyecto
Alcance	Definir el alcance · Definir las actividades · Controlar y verificar el alcance
Tiempo	Establecer la secuencia de actividades · Estimar la duración · Desarrollar el cronograma · Controlar el cronograma
Costes	Estimar los costos · Elaborar los presupuestos · Controlar los costos
Calidad	Planificar la calidad · Aseguramiento de calidad · Control de calidad
Recursos	Planificar los recursos · Controlar los recursos
Personal	Definir el equipo del proyecto · Gestionar el equipo
Comunicación	Planificar las comunicaciones · Gestionar la información y los interesados
Riesgos	Planificar la gestión · Identificar · Analizar · Planificar la respuesta · Controlar
Adquisiciones	Planificar las adquisiciones · Planificar la contratación · Solicitar respuesta a proveedores · Seleccionar proveedores · Administrar el contrato · Cerrar el contrato

Tres detalles que se preguntan. **Definir las actividades** pertenece a **Alcance**, no a Tiempo, aunque suene a cronograma. **Desarrollar el cronograma** debe identificar explícitamente el **camino crítico**, que es el de mayor duración en la red de actividades, y de ahí salen las actividades críticas y los hitos. Y **definir el equipo del proyecto** consiste en determinar **roles y responsabilidades**; no incluye estimar el esfuerzo por rol ni controlar el cronograma. Además, *elaborar los presupuestos* debe incluir **reservas para contingencias**, y *cerrar el proyecto* vale tanto para el completado como para el cancelado.

10.5 La EDT, las precedencias y el esfuerzo

La secuencia de trabajo es siempre la misma: se arma la **EDT**, se identifican las **tareas típicas** del ciclo de vida elegido, y se establecen las **órdenes de precedencia** entre ellas.

EDT (estructura de desglose del trabajo): descomposición del proyecto en **paquetes de trabajo** manejables, que sirve de base para asignar esfuerzo, calendario y responsables.

La EDT **lista** las tareas —verbos concretos: "Analizar módulo A", no "Análisis"—; el orden y las dependencias van en el cronograma. Al leer una red de tareas, una **predecesora directa** es la inmediatamente anterior: si A precede a B y B precede a C, A **no** es predecesora directa de C, aunque tenga que ocurrir antes. Y los **módulos independientes arrancan en paralelo**: si un producto se compone de tres módulos independientes, las tareas que dan inicio al proyecto son **tres**, una de análisis por módulo.

ATENCIÓN Esfuerzo y duración no son lo mismo y no se suman igual. El **esfuerzo** se mide en días-persona u horas-persona y **se suma**. La **duración** se mide en días o meses y **no se suma**: sale del **camino crítico**, porque las tareas independientes corren en paralelo. Dos tareas de 4 y 5 semanas suman siempre 9 semanas de esfuerzo, pero duran 9 si van en secuencia y 5 si van en paralelo. Por eso reordenar tareas cambia la duración, no el esfuerzo. Cuando un ejercicio pide esfuerzo, hay que leer la columna de esfuerzo y convertir: 1 día-persona = 8 horas.

Hito: evento en el que se planea una entrega relevante o se requiere una decisión.

Un hito bien redactado es **un producto terminado o una aprobación**: "especificación de requisitos aprobada", "sistema instalado y aceptado". Los **puntos de control**, en cambio, se ubican **en función de los riesgos**: más controles donde es más probable que algo salga mal, y siempre antes de que la etapa termine, con margen para corregir.

11. La estimación del esfuerzo

Estimar es decidir cuánto va a costar algo que todavía no existe. La materia presenta cuatro métodos, que no compiten entre sí: se usan en momentos distintos y con información distinta.

El **valor esperado**, o **técnica de tres puntos**, estima cada tarea en tres escenarios —optimista, normal y pesimista— y los combina en un único valor. La ponderación más difundida le da peso cuatro al escenario normal: $(0 + 4N + P) / 6$ (conocimiento general). Es rápida y sirve cuando la incertidumbre está acotada.

Delphi es una estimación **grupal e independiente de expertos**: cada uno vuelca su perspectiva en números sin ver la del resto. Es **iterativa**, y en cada vuelta se suma información buscando la **convergencia**. Sirve cuando no hay datos históricos pero sí gente con experiencia.

Los **puntos de función** miden el tamaño del software desde una perspectiva funcional, independiente de la tecnología. Son una medida **indirecta del tamaño**, no del esfuerzo: el esfuerzo se deriva después. Los **puntos de historia** son la estimación relativa de las metodologías ágiles.

Un consejo que la cátedra repite: **tomar nota de las decisiones tomadas**, sobre todo en la etapa de estimación. Es lo que después permite explicar un desvío en PMC en lugar de improvisar una justificación.

11.1 Del tamaño al esfuerzo

Los puntos función, las líneas de código o la cantidad de requerimientos miden **tamaño**. Para llegar a horas hace falta un **indicador de productividad** —puntos función por mes, horas por punto función— sacado de **proyectos similares**: promediar proyectos que no se parecen da un indicador que no sirve para ninguno.

ATENCIÓN **Tamaño no es esfuerzo**, y en CMMI son prácticas distintas. "Se cuentan las entradas, salidas, consultas y ficheros y se obtienen 320 PF" es **PP SP 1.2**. "Con la productividad histórica de 12 PF por mes, el desarrollo lleva 26,7 meses-persona" es **PP SP 1.4**.

11.2 La distribución 40-20-40

Es una regla de reparto del esfuerzo total de un proyecto: **40 % análisis y diseño** —de los cuales 10 a 15 puntos van al análisis y 25 a 30 al diseño—, **20 % codificación** y **40 % pruebas**.

La lectura que importa no es el número exacto sino su consecuencia: **codificar es apenas la quinta parte del proyecto**, y se prueba tanto como se analiza y diseña junto.

11.3 Análisis de puntos función

El APF mide el tamaño del software **entregado al usuario**. Como los puntos función de un sistema no cambian con el lenguaje ni la plataforma, sirven para **comparar productividad** entre herramientas u organizaciones, y para **seguir los cambios de alcance**: si los puntos función medidos van creciendo de una fase a otra, hubo cambios de alcance. La guía lo describe en dos etapas: primero se identifican y clasifican las funciones, y después se ponderan y se ajusta el total.

Identificar y clasificar. Se fija el **límite del sistema**, que determina qué queda adentro y qué es externo, y se reconocen cinco componentes: **entradas** (datos que cruzan el límite hacia adentro), **salidas** (datos que lo cruzan hacia afuera), **consultas** (combinación de entrada y salida para obtener datos), **ficheros lógicos internos** o FLI (datos que residen dentro de la aplicación y son actualizados por las entradas) y **ficheros de interfaz externos** o FIE (datos que residen fuera y son mantenidos por otra aplicación).

Ponderar. La complejidad se establece por la diversidad de atributos en tipo y cantidad: en las transacciones se cruzan los **ficheros referenciados** con los **tipos de datos**; en los ficheros, los **elementos de registro** con los tipos de datos. Con más de cada cosa, la complejidad sube de baja a media y a alta. Cada componente se multiplica por el peso de su complejidad:

COMPONENTE	BAJA	MEDIA	ALTA
Entradas	3	4	6
Salidas	4	5	7
Consultas	3	4	6
Ficheros internos (FLI)	7	10	15
Ficheros externos (FIE)	5	7	10

La suma da los **PFD**, puntos función sin ajustar.

Ajustar. El factor de ajuste incorpora características no funcionales del entorno. Se califican **14 características** de 0 a 5 —comunicaciones de datos, procesamiento distribuido, rendimiento, utilización masiva, tasa de transacción, entrada de datos on-line, eficiencia para el usuario, actualización on-line, procesamiento complejo, reutilización, facilidad de instalación, facilidad de operación, puestos múltiples y facilidad de cambio—. La suma de los 14 valores es el **TDI**, que va de 0 a 70:

$$\begin{aligned}\text{Factor de ajuste} &= (65 + \text{TDI}) / 100 \\ \text{PF ajustados} &= \text{PFD} \times \text{Factor de ajuste}\end{aligned}$$

Con el TDI entre 0 y 70, el factor va de **0,65 a 1,35**: el ajuste puede mover el tamaño estimado hasta un 35 % para cada lado. Por ejemplo, 200 PFD con un TDI de 42 dan un factor de 1,07 y **214 PF**.

11.4 Puntos función de una mejora

La guía avanzada mide también **mejoras**, es decir, cambios sobre un sistema que ya existe. Cada función afectada mide sus puntos función sin ajustar multiplicados por un **factor de impacto**: una función **añadida** vale 1; una **eliminada**, 0,4; una **modificada**, entre 0,25 y 1,5 según el porcentaje de cambio, que sale de una tabla. En el ejemplo de la guía, un informe de 5 PF con un factor de impacto de 0,75 mide **3,75 PF** de mejora.

$$\text{PFM} = (\text{añadidos} + \text{modificados}) \times \text{FA después} + \text{eliminados} \times \text{FA antes}$$

ATENCIÓN El factor de ajuste de **después** de la mejora se aplica a lo **añadido y modificado**; el de **antes**, a lo **eliminado**.

Las siglas del glosario, que también entra: **PFD** (puntos función sin ajustar), **PFDM** (sin ajustar de la mejora), **PFM** (de la mejora) y **PFP** (de las pruebas, que miden aparte las funciones del sistema mejorado que intervienen en las pruebas). Un **fichero referenciado** es un FLI leído o modificado por una transacción, o un FIE leído por ella.

12. La gestión de riesgos (RSKM)

Todo proyecto convive con cosas que pueden salir mal: el único que conoce la base de datos renuncia, el proveedor entrega tarde, el cliente descubre a mitad de camino que necesitaba otra cosa. En nivel 2 el proyecto ya **identifica** riesgos al planificar (PP) y los **monitoriza** durante la ejecución (PMC), pero en general **reacciona** cuando se materializan. La gestión de riesgos es el paso siguiente: prepararse, prevenir y mitigar de forma **sistémica**, porque detectar un riesgo temprano es más fácil, más barato y menos dañino que corregir sus efectos en una fase tardía.

Propósito de RSKM: identificar los problemas potenciales **antes de que ocurran**, para que las actividades de tratamiento de riesgos puedan planificarse e invocarse según sea necesario, para mitigar los impactos adversos en el logro de los objetivos.

Es un proceso **continuo** que mira hacia adelante, y necesita un entorno de **divulgación libre**: se identifican riesgos, no culpables. Es de **nivel 3**; la carátula de la traducción que dice nivel 2 es una errata.

12.1 Qué es un riesgo

Riesgo de un proyecto: evento o condición **incierto** que, si ocurre, tiene un efecto **positivo o negativo** sobre al menos un objetivo del proyecto: tiempo, coste, alcance o calidad.

Tiene tres componentes: un **evento**, su **probabilidad** y su **consecuencia** o impacto. Y la definición trae una sorpresa que se pregunta: un riesgo **también puede ser una oportunidad**. Los riesgos pueden ser **conocidos**, que se identifican y planifican, o **desconocidos**, que se cubren con una reserva; e **internos**, controlables por el equipo, o **externos**, como una ley nueva o una huelga.

12.2 Metas y prácticas

CMMI divide la gestión de riesgos en tres partes, que son sus tres metas: prepararse, identificar y analizar, y mitigar.

- **SG 1 Preparar la gestión de riesgos.** SP 1.1 Determinar las **fuentes y las categorías** de los riesgos · SP 1.2 Definir los **parámetros** de los riesgos (probabilidad, consecuencia y **umbrales**) · SP 1.3 Establecer una **estrategia** de gestión de riesgos.
- **SG 2 Identificar y analizar los riesgos.** SP 2.1 **Identificar** los riesgos · SP 2.2 **Evaluar, categorizar y priorizar** los riesgos.
- **SG 3 Mitigar los riesgos.** SP 3.1 Desarrollar los **planes de mitigación** · SP 3.2 **Implementar** los planes de mitigación.

ATENCIÓN El orden es **identificar** → **analizar** → **priorizar**. Es falso que "el análisis de riesgos es previo a su identificación": un riesgo tiene que estar identificado antes de poder analizarse. Y los riesgos se identifican **desde la planificación** del proyecto.

12.3 Umbrales, mitigación y contingencia

Los **umbrales** son lo que separa "anotar el riesgo" de "actuar". Con ellos, un riesgo puede quedar **aceptado**, cuando es demasiado bajo para justificar una mitigación formal o no hay forma de reducirlo —y entonces la razón **debe documentarse**—, o **vigilado**, cuando hay límites objetivos y documentados que, al superarse, activan el plan de mitigación o el de contingencia.

ATENCIÓN Por eso es **falso** que "cualquier desvío en un riesgo habilita actividades de tratamiento": se actúa al **superar el umbral**. Es la misma lógica del desvío significativo de PMC.

La **mitigación** actúa **antes** de que el riesgo ocurra, para reducir su probabilidad o su impacto. La **contingencia** responde **después**, cuando el riesgo ocurre a pesar de todo. Las dos se planifican de antemano. Ante el riesgo de que renuncie el único DBA, capacitar desde ahora a un segundo DBA es **mitigación**; dejar previsto que, si renuncia, se contrata a una consultora es **contingencia**.

Para decidir qué hacer con cada riesgo, las dos fuentes de la materia usan listas que no coinciden, y eso se presta a trampas. Para **CMMI**, aceptar es reconocer el riesgo **sin tomar ninguna acción**, y transferir es **reasignar requerimientos**. Para **INTECO**, la aceptación puede ser pasiva o **activa** —con una reserva y un plan de contingencia—, y transferir es trasladar el impacto a un **tercero**: seguros, garantías, contratos. INTECO agrega estrategias para las oportunidades: explotar, compartir y mejorar.

12.4 El proceso y las reservas según INTECO

La guía práctica de gestión de riesgos baja todo esto a seis actividades, que se actualizan durante todo el proyecto, y en todas la responsabilidad final es del **jefe de proyecto**. Se arma el **plan de gestión de riesgos**, que dice **cómo** se va a gestionar —ojo: **no contiene los planes de respuesta** de cada riesgo—. Se **identifican** los riesgos en un **registro de riesgos**, con técnicas como la tormenta de ideas, **Delphi** o las listas de control. Se **analizan**: el análisis **cualitativo** cruza probabilidad e impacto en una matriz que da la prioridad, y el **cuantitativo** calcula el **valor esperado** = **probabilidad** × **impacto**. Se **planifica la respuesta**, con un **propietario** por riesgo y los **riesgos residuales** que quedan después de ella. Y se **controla** y se **cierra** registrando las lecciones aprendidas.

Responder a un riesgo cuesta, y la guía distingue de dónde sale ese dinero. La **reserva de contingencia** cubre el valor esperado de los riesgos **aceptados** y **residuales**, es decir, de riesgos conocidos. La **reserva de gestión** cubre los **desconocidos**. Y el costo de evitar, transferir o mitigar **no va a ninguna reserva**: va al **presupuesto**, porque se sabe cuánto cuesta. Un riesgo con probabilidad 0,70 y un impacto de \$20.000 tiene un valor esperado de \$14.000; si se acepta, ese monto suma a la reserva de contingencia.

12.5 Riesgos en PP, PMC y RSKM

Las tres áreas hablan de riesgos, y es uno de los cortes más finos al reconocer el área de un enunciado.

ACTIVIDAD DEL ENUNCIADO	ÁREA / PRÁCTICA
Identificar riesgos para armar el plan y obtener el acuerdo de los interesados sobre ellos	PP SP 2.2
Revisar los riesgos contra los del plan y comunicar su estado	PMC SP 1.3
Definir fuentes, parámetros, umbrales o la estrategia; armar o ejecutar los planes de mitigación	RSKM

Dos ítems de la práctica oficial para el AD muestran el corte. "Informar a los stakeholders que aumentaron las chances, respecto de lo planificado, de que un módulo no pueda desarrollarse; aún es posible resolverlo" es **PMC SP 1.3**: se compara contra el plan y el riesgo todavía no ocurrió; si ya hubiera ocurrido, sería un problema, PMC SP 2.1. "Previo al inicio del desarrollo, revisar con el cliente si están de acuerdo con el impacto documentado para los riesgos" es **PP SP 2.2**: es planificación.

En los verdadero o falso del tipo "¿es un riesgo posible en este proyecto?", un riesgo con impacto positivo o de baja probabilidad **sigue siendo un riesgo posible**; la afirmación es falsa sólo si el enunciado **lo descarta**. Y en formato BP, una software factory que registra los riesgos recién cuando el problema aparece **sí** tiene un problema de calidad: es la postura reactiva que RSKM viene a superar.

13. Los ciclos de vida

Elegir un ciclo de vida es decidir **cuándo** se hace cada cosa, y esa decisión determina cuándo se detectan los defectos y cuánto cuesta corregirlos.

Ciclo de vida: conjunto de fases por las que pasa el sistema **desde que nace la idea inicial hasta que el software es retirado o reemplazado**.

Quién lo decide depende del alcance: elegir el ciclo de un proyecto es **PP SP 1.3**; establecer las opciones de ciclos de vida de toda la organización es **OPD SP 1.2**, y fijar con qué criterio se elige entre ellas, **OPD SP 1.3**.

CICLO	CÓMO FUNCIONA	CUÁNDO CONVIENE
Cascada	Cada etapa espera a que termine la anterior. Las pruebas van al final	Requerimientos conocidos y estables ; proyectos cortos; mantenimiento correctivo corto. Es el que menos sirve con requerimientos inestables
Modelo en V	Cada etapa de desarrollo tiene su nivel de prueba asociado , planificado desde el comienzo	Cuando se quiere detectar defectos temprano sin abandonar la estructura secuencial
Incremental	Secuencias lineales escalonadas; cada una entrega un incremento operativo . Cada incremento es una mini-cascada completa	Cuando hay que salir a producción antes de tener todo terminado
Iterativo	Varias pasadas sobre el producto; en cada una se refinan los mismos requisitos	Requerimientos poco claros o no estabilizados
Espiral	Vueltas con análisis de riesgos explícito; las características evolucionan con prototipos	Proyectos largos, caros o riesgosos, cuando el equipo no puede especificar por adelantado
Prototipo	Diseño rápido de lo visible para el usuario, evaluación del cliente, refinamiento	El cliente define objetivos generales pero no los requisitos detallados . Reduce el riesgo de subestimar el esfuerzo

La desventaja crítica de la cascada es que concentra las pruebas al final, de modo que los defectos se detectan cerca de la implementación, que es donde más caro sale corregirlos. El modelo en V nace como respuesta: la aceptación se apoya en los requisitos de usuario, la prueba de sistema en la especificación, la de integración en el diseño, y las unitarias en el código.

Incremental, iterativo y espiral se confunden porque los tres repiten. En el **incremental** cada incremento es un **pedazo distinto** del producto que se pone en funcionamiento; en el **iterativo** cada vuelta retoma **los mismos** requisitos para mejorarlos. Y entre incremental y espiral la diferencia es la **incertidumbre**: el incremental parte de que se conoce el problema y lo divide; el espiral la asume alta y la gestiona con análisis de riesgos.

ATENCIÓN Como cada incremento es una cascada completa, **el análisis se hace en todos los incrementos y las pruebas de sistema también**, no solamente en el último. Y como cada incremento se entrega, un incremental de cuatro incrementos tiene como mínimo **cuatro releases**, mientras que una cascada tiene **uno** (capítulo 20).

En los ejercicios el ciclo hay que **justificarlo con elementos del enunciado**. Según el cuestionario de la cátedra, lo que incide en la elección es la **estabilidad de los requerimientos** y la **necesidad de poner algo en producción antes de terminar**; el presupuesto, la cantidad de recursos y el lenguaje, no. Con requisitos incompletos, la cascada es el menos adecuado: el riesgo de no haberlos entendido se materializa al final, cuando el cliente ve el producto. Un dato que se tomó: en una cascada, el **manual de usuario** se confecciona en la fase de prueba, a partir de las interfaces y los casos de uso.

14. Los requerimientos y su desarrollo

Según el NIST, que cita la guía de requisitos de la cátedra, los requisitos **incompletos, imprecisos o contradictorios** causan cerca del **70 % de los defectos** de una aplicación. El problema no suele ser que después no se puedan corregir, sino que, por falta de tiempo, el equipo **se precipita o supone**, y el costo del producto se multiplica. De ahí salen dos ideas que sirven para justificar respuestas: los requisitos tienen que ser **entendidos por todas las partes antes de construir**, y **no se congelan**: el producto evoluciona y sus requisitos también, así que la evolución hay que gestionarla, no negarla.

Requisito: algo que el producto debe hacer o una característica que debe tener; una condición o capacidad que el sistema tiene que cumplir. Se escribe en forma **tecnológicamente neutra**: dice **qué**, no **cómo**.

La **ingeniería de requisitos** tiene dos mitades. El **desarrollo** produce los requisitos; la **gestión** administra los que ya existen: sus cambios y su consistencia con el resto del proyecto. En CMMI son dos áreas: **RD**, de **nivel 3**, y **REQM**, de **nivel 2**. Este capítulo trata la primera mitad; el siguiente, la segunda. La guía de INTECO dice *requisito* y la traducción del CMMI dice *requerimiento*: son lo mismo.

14.1 Tipos de requisitos

Por nivel de abstracción, hay requisitos **de negocio** —objetivos, visión y alcance—, **de usuario** —las tareas que el sistema ejecuta cuando el usuario opera con él— y **de sistema** —las funcionalidades que satisfacen a los anteriores, base del diseño y de las pruebas—. A eso se suman las **restricciones**, que limitan las opciones del diseñador.

La distinción que más se pregunta es la de **funcionales**, que dicen **qué debe hacer** el producto, y **no funcionales**, que dicen qué **cualidades** debe tener: rapidez, fiabilidad, seguridad, usabilidad. La guía lo dice con una imagen: los funcionales hacen que el producto realice el trabajo; los no funcionales le dan **carácter** al trabajo.

ATENCIÓN Un resumen de alumnos dice que los no funcionales "no son requeridos pero son deseables". La guía dice lo contrario: son **tan importantes como los funcionales**, y a veces deciden la aceptación. "Los no funcionales son opcionales" es **falso**.

Lo que el cliente dice no viene ordenado, y el analista tiene que clasificarlo. Si el cliente describe una forma específica de interactuar con el sistema, eso es **una solución sugerida, no un requisito**.

14.2 El desarrollo de requisitos según la guía

La guía organiza el desarrollo en cuatro actividades.

La **obtención** busca los requisitos resolviendo las diferencias entre los involucrados. Bien hecha, produce requisitos completos, consistentes, **identificados de forma única**, priorizados, **claros y no ambiguos**, y **testeables**, para poder verificarlos y validarlos después. El analista tiene que sacar a la luz también lo que el usuario **no sabe que necesita**, porque lo tiene tan internalizado que lo olvida: capturarlo ahora es mucho más barato que cuando aparece con el producto en uso. Las técnicas son las entrevistas, las reuniones, la observación, los cuestionarios —útiles cuando la gente es mucha o importa el anonimato—, el brainstorming, los casos de uso y los prototipos.

La **definición** los escribe. Las reglas son concretas: sin pronombres, con cuidado en los adjetivos y **sin "debería"**, que da a entender que el requisito es opcional; en lenguaje de negocio, y con un **glosario** que evoluciona con el proyecto.

La **verificación** controla cada requisito antes de que entre a la especificación, en una **puerta de calidad** que custodian el responsable del análisis y el **técnico de pruebas**. Ahí el usuario agrega los **criterios de aceptación**, y se evitan las **fugas de requisitos**: los que aparecen en la especificación sin que nadie sepa de dónde vienen.

La **revisión de la especificación** controla el **conjunto**: que no falte ningún tipo de requisito y que no haya **conflictos** entre ellos —dos requisitos están en conflicto si la solución de uno impide implementar el otro—. La **priorización** la hace el cliente por **valor**, con la información de **costo, dificultad y riesgo** que le da el desarrollador; si el cliente no logra priorizar, decide el jefe de proyecto.

Las responsabilidades quedan así: el **jefe de proyecto** es responsable de todo el proceso; el **cliente aprueba** los requisitos; el **CCB aprueba los cambios** (capítulo 15); y **SQA revisa**, pero no aprueba.

Un **prototipo** es una simulación de los requisitos, y rinde cuando los usuarios no saben bien lo que necesitan. El **horizontal** muestra todas las pantallas casi sin lógica, para aclarar el alcance; el **vertical** desarrolla pocas funciones en profundidad. En ningún caso **reemplaza la especificación escrita**: la complementa.

14.3 RD – Desarrollo de requerimientos

Propósito de RD: producir y analizar los requerimientos **de cliente, de producto y de componente del producto**.

Los tres tipos existen porque el cliente habla en sus términos: éstos son los **requerimientos de cliente**. Para diseñar hay que traducirlos a términos técnicos, y así nacen los **de producto y de componentes**.

- **SG 1 Desarrollar los requerimientos de cliente.** SP 1.1 **Obtener las necesidades** · SP 1.2 **Desarrollar** los requerimientos de cliente (consolidar lo relevado, resolver conflictos entre interesados).
- **SG 2 Desarrollar los requerimientos de producto.** SP 2.1 Establecer los requerimientos de producto y de componentes · SP 2.2 **Asignar** los requerimientos de componentes · SP 2.3 Identificar los requerimientos de **interfaz**.
- **SG 3 Analizar y validar los requerimientos.** SP 3.1 Establecer los conceptos operativos y los **escenarios** · SP 3.2 Establecer una definición de la funcionalidad requerida · SP 3.3 **Analizar** los requerimientos (que sean completos, factibles y **verificables**: acá cae el requisito vago o no medible) · SP 3.4 Analizar para alcanzar el **equilibrio** con costo, plazo y riesgo · SP 3.5 **Validar** los requerimientos.

ATENCIÓN En RD, toda opción en la que **el equipo decide solo** es incorrecta: elegir los dispositivos, completar con métricas estándar de la industria, "lo resolvemos en las pruebas". RD exige **confirmar con el cliente**. En el AD de 2024, ante un cliente que pedía acceso desde dispositivos móviles sin decir cuáles, la correcta fue definir una lista de dispositivos y sistemas operativos y **confirmarla con el cliente**. Ante requisitos de usabilidad vagos, la correcta fue reunirse con el cliente para definir **métricas** y actualizar la documentación.

RD tiene dos fronteras que se preguntan. **Con REQM:** RD documenta las relaciones entre requerimientos al refinarlos, pero **mantener la trazabilidad** es REQM SP 1.4. **Con VAL:** RD SP 3.5 valida **los requerimientos**, temprano y sobre prototipos o simulaciones, mientras que VAL valida **el producto**; probar el sistema con el cliente es VAL. Y como RD define desde el relevamiento las restricciones para verificar y validar, "la validación recién empieza al final del proyecto" es falso.

15. La gestión de requerimientos y de los cambios

Una vez definidos los requisitos —e incluso con el sistema ya implantado— aparecen necesidades nuevas, correcciones y mejoras. El cambio no es una falla del relevamiento: es inherente al software. La falla es no gestionarlo. El cliente le pide un ajuste por mail al programador, el programador lo hace esa tarde, y tres meses después nadie sabe por qué el sistema calcula distinto de lo que dice la especificación, ni qué otras partes tocó ese cambio.

La **gestión de requisitos** es el conjunto de actividades para **identificar, controlar y seguir** los requisitos y sus cambios, y asegurar la **consistencia** entre los requisitos y el sistema construido. Su actividad más importante es gestionar los **cambios**, y trabaja sobre una línea base.

Línea base: conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante un procedimiento formal de control de cambios**.

Cambiar la línea base de requisitos es, por definición, un **cambio de alcance**.

15.1 El orden ante un cambio

El proceso de la guía para cada petición de cambio tiene un orden que se pregunta una y otra vez. **Primero se registra y se evalúa el impacto:** con la matriz de trazabilidad se ve qué otros requisitos, módulos y casos de prueba se afectan, y cuánto cuesta el cambio. **Después se valora:** si el impacto es asumible, se acepta; si no, se **negocia con el cliente**; si se rechaza, queda registrado. **Recién entonces** se modifican todos los productos afectados —la especificación, la matriz y, si hace falta, el plan—, se establece una **nueva línea base** y se obtiene la aprobación del cliente.

ATENCIÓN Toda opción que **implemente antes de evaluar el impacto** es incorrecta. En el AD de 2024, ante un cliente que en plena implementación pedía restricciones horarias que afectaban a varios subsistemas, la correcta fue analizar cómo afectan a los otros subsistemas y actualizar la matriz de trazabilidad **antes de proceder**. La trampa era "registrar el cambio, actualizar la matriz y **comenzar la implementación**": parece REQM, pero se saltea la evaluación.

En el mismo parcial, el cliente de un comercio electrónico quería registrar la navegación de los usuarios, lo que implicaba cumplir normas de protección de datos. El primer paso correcto fue **consultar con legales y privacidad** y reflejar el impacto en la matriz: las leyes son fuente de requisitos y de cambios como cualquier otra.

15.2 La trazabilidad

Trazabilidad bidireccional: asociación **en ambos sentidos** entre el requisito y su **origen** (hacia atrás) y entre el requisito y el diseño, el código y las pruebas (hacia adelante).

Hacia atrás responde **de dónde viene** el requisito y si tiene una fuente válida; hacia adelante, **qué se rompe si lo cambio** y si está cubierto por pruebas. Sirve, sobre todo, para **evaluar el impacto de un cambio**. Y tiene un costo: si actualizarla es muy trabajoso, los desarrolladores cambian el código sin actualizar los requisitos, que se vuelven inútiles para probar.

ATENCIÓN ¿Qué área **establece y mantiene** la trazabilidad entre los requerimientos y los otros productos de trabajo? REQM, SP 1.4.

15.3 REQM – Gestión de requerimientos

Propósito de REQM: gestionar los requerimientos de los productos y componentes del proyecto, e **identificar inconsistencias** entre esos requerimientos y los **planes y productos de trabajo** del proyecto.

Es un área de **nivel 2** con una sola meta, **SG 1 Gestionar los requerimientos**, y gestiona **todos** los requerimientos que el proyecto recibe o genera, vengan de donde vengan.

PRÁCTICA	CÓMO SE RECONOCE EN UN ENUNCIADO
SP 1.1 Obtener una comprensión de los requerimientos	Entender con quien lo pidió qué significa; canales oficiales, criterios de aceptación
SP 1.2 Obtener el compromiso sobre los requerimientos	Acordar con quienes implementan, o con el Sponsor, el impacto sobre un compromiso
SP 1.3 Gestionar los cambios de los requerimientos	Documentar la razón del cambio; " se está chequeando cómo afecta"
SP 1.4 Mantener la trazabilidad bidireccional	"Actualizar la matriz de trazabilidad "
SP 1.5 Identificar las inconsistencias entre el trabajo y los requerimientos	"El plan o el caso de uso no coincide con el requerimiento"

La diferencia entre SP 1.1 y SP 1.2 es **con quién**. La comprensión se logra con el **proveedor** del requerimiento: el cliente o el usuario que lo pide. El compromiso, con los **participantes del proyecto** que lo van a implementar. Acordar con el Sponsor posponer tres semanas la implementación para incorporar unos cambios es **REQM SP 1.2**: se negocia el impacto de un cambio sobre un compromiso existente.

15.4 Las solicitudes de cambio

La gestión de requisitos dice **qué** decidir ante un cambio; la **gestión de solicitudes de cambio** de RUP dice **cómo** se tramita, para que todo cambio pase por un procedimiento estándar, deje registro y se haga con efecto predecible.

Solicitud de cambio (CR): producto de trabajo enviado formalmente para rastrear las solicitudes de los interesados —funciones nuevas, mejoras, **defectos**, requisitos cambiados— con su estado e historial durante todo el ciclo de vida.

CCB (panel de control de cambios): grupo que supervisa el proceso de cambio, con representantes de **todas las partes interesadas**. En un proyecto chico puede ser **una sola persona**.

La reunión del CCB decide si cada CR es **válida** y si entra en el release actual. El camino feliz de una CR es **Enviado → Abierta → Asignada → Resuelta → Verificada → Cerrada**; los desvíos son **Pospuesta** (válida pero fuera del release actual), **Duplicada**, **Rechazada**, **Más información** y **La prueba ha fallado**. Sólo la reunión del CCB puede abrir una CR, y sólo el administrador de revisión del CCB puede cerrarla.

En CMMI, este proceso es **CM** (capítulo 20): **SP 2.1 Seguir las peticiones de cambio** y **SP 2.2 Controlar los elementos de configuración**. Las solicitudes de cambio no son de REQM, y no tratan sólo requisitos: también **defectos**.

15.5 REQM, RD o CM: cómo decidir

Las tres áreas giran alrededor de los requisitos, y separarlas es de lo más preguntado de la unidad. Dos pasos resuelven casi todo.

Primer paso: ¿el requisito ya está acordado? Si todavía se está relevando, aclarando, completando o validando, es **RD**. Si ya está en la línea base, seguí al segundo paso.

Segundo paso: ¿qué se hace con el requisito acordado?

SITUACIÓN DEL ENUNCIADO	ÁREA / PRÁCTICA
Llega un cambio y todavía se evalúa cómo afecta	REQM SP 1.3
Actualizar o consultar la matriz de trazabilidad	REQM SP 1.4
El plan o los casos de uso no coinciden con los requisitos vigentes	REQM SP 1.5
El cambio ya se decidió aceptar y se tramita: seguir la petición, ver qué modificar en los casos de uso	CM SP 2.1
Revisar y acordar formalmente un conjunto de artefactos como base del desarrollo	CM SP 1.3
Alguien externo al proyecto detecta cambios hechos sin seguir el procedimiento y escala	PPQA SP 2.1

El ejemplo canónico viene del cuestionario de la cátedra. "El cliente pidió una variante de descuento no acordada; en este momento se está chequeando con el contador cómo afecta al esquema impositivo" es **REQM SP 1.3**. "Ya se decidió aceptar la variante; ahora se chequean las modificaciones a realizar en los casos de uso" es **CM SP 2.1**.

ATENCIÓN El análisis de impacto aparece **en las dos áreas**. Lo que decide es **el momento**: si todavía se evalúa si conviene, REQM; si ya se aceptó y está en trámite, CM. Y decide el **contenido** del cambio, no quién lo pide: un cambio interno que **no altera ningún requisito** —el equipo refactoriza o corrige un defecto— se controla en **CM**; si altera un requisito, es REQM aunque lo proponga el equipo. La regla de alumnos "REQM trata los cambios que pide el cliente" sirve como atajo, porque casi siempre coincide.

16. Verificación y validación

Un sistema puede fallar de dos maneras muy distintas. Puede **no hacer lo que dice su especificación**: el recargo está mal calculado, una pantalla acepta un dato que debería rechazar. O puede hacer **exactamente lo que dice su especificación y aun así no servirle al cliente**, porque la especificación quedó mal relevada. Son dos problemas diferentes, se detectan con preguntas diferentes, y CMMI les dedica dos áreas de proceso de **nivel 3**.

16.1 La distinción

Es la pareja de conceptos más importante de la unidad y la más fácil de confundir, porque ambos son procesos de evaluación de productos, se ejecutan frecuentemente **de forma concurrente** y pueden compartir parte del entorno.

	VERIFICACIÓN (VER)	VALIDACIÓN (VAL)
Qué asegura	Que los productos de trabajo seleccionados cumplen sus requerimientos especificados	Que el producto se ajusta a su uso previsto cuando se sitúa en su entorno previsto
La pregunta	¿Estoy construyendo correctamente el producto?	¿Estoy construyendo el producto correcto ?
Contra qué contrasta	La especificación	La necesidad del usuario
Quién interviene	El equipo: es una cuestión interna	El cliente o usuario
Técnicas típicas	Revisiones entre pares , inspecciones, pruebas contra la especificación	Revisión de requisitos con el cliente , demostración de prototipos, pruebas de aceptación , piloto

ATENCIÓN El criterio que resuelve cualquier caso es **contra qué se contrasta y quién participa**, no qué tan terminado está el producto ni en qué fase ocurre. Unas pruebas de sistema ejecutadas por el equipo de QA sobre el producto terminado son **verificación**, porque se contrastan contra la especificación. Unas pruebas de aceptación con clientes reales son **validación**. Y revisar los requisitos **con el cliente** también es validación, aunque no se ejecute nada.

16.2 El criterio rápido para un enunciado

En el parcial casi nunca piden la definición: describen una actividad y preguntan el área. La regla tiene tres salidas. Si aparece el **cliente o usuario**, o se prueba **el uso real**, es **VAL**. Si se controla **un artefacto contra otro artefacto del proyecto** —casos de uso contra minutas, etiquetas contra el glosario— sin el cliente, es **VER**. Y si se controla contra un **estándar o procedimiento de la organización**, es **PPQA** (capítulo 21). Un ejemplo de clase, sobre un mismo caso de uso: "no se usó la etiqueta de tiempo que pide el estándar" es PPQA; "se usó *durante* cuando correspondía *reemplaza*" es VER, porque el contenido no satisface lo especificado.

El AD de 2025 lo preguntó con **diagramas de conjuntos**: **S**, la especificación; **P**, el producto implementado; **Test**, las pruebas ejecutadas. El planteo con Test casi entero **dentro de S** es **VER**: los casos salen de la especificación y miran si el producto la cumple. El planteo con Test casi entero **dentro de P** es **VAL**: se prueba el producto real en su entorno de uso, incluso comportamiento que la especificación no contempla.

Queda una frontera con **RD SP 3.5 Validar los requerimientos** (capítulo 14), que valida los requerimientos, temprano y sobre prototipos. **VAL** valida **el producto o sus componentes**: elegir los casos de uso que entran en la prueba de aceptación, o probar con el cliente en un piloto, es VAL.

ATENCIÓN Cuatro trampas que se repiten. "**Dinámica = validación**" es falso: las pruebas de integración y de sistema son dinámicas y son VER. **Una revisión entre pares es siempre VER**; una revisión **con el cliente** es VAL. **La inspección es una técnica, no un área**: la usa VER y también PPQA. Y aunque CMMI nombre la prueba de aceptación entre los métodos de VER, en el examen se tomó **aceptación** → **VAL**: la ejecuta el cliente para confirmar que el producto le sirve.

16.3 Metas y prácticas

VERIFICACIÓN (VER)

SG1 Preparar la verificación — SP1.1 Seleccionar los productos de trabajo a verificar · SP1.2 Establecer el entorno · SP1.3 Establecer procedimientos y criterios

SG2 Realizar revisiones entre pares — SP2.1 Preparar · SP2.2 Llevar a cabo · SP2.3 Analizar los datos

SG3 Verificar los productos seleccionados — SP3.1 Realizar la verificación · SP3.2 Analizar los resultados

VALIDACIÓN (VAL)

SG1 Preparar la validación — SP1.1 Seleccionar los productos a validar · SP1.2 Establecer el entorno · SP1.3 Establecer procedimientos y criterios

(VAL no tiene esta meta)

SG2 Validar el producto o los componentes — SP2.1 Realizar la validación · SP2.2 Analizar los resultados

ATENCIÓN Las **revisiones entre pares** existen sólo en VER. Si el enunciado describe una revisión entre pares y la opción dice VAL/SP 2.1 Preparar las revisiones entre pares, la respuesta es **NINGUNA**: hay que leer la sígla, no sólo el nombre de la práctica.

Para elegir la práctica sirve el **tiempo verbal**. En futuro —"será chequeada", "se distribuirá"— se está **preparando** (SG 1, o SP 2.1 si es una revisión entre pares); en presente —"está chequeando"— se está **realizando**; y "ya se documentaron los hallazgos, ahora se almacenan" es **analizar los datos**.

LO QUE DESCRIBE EL ENUNCIADO

PRÁCTICA

Decidir qué productos se verifican y con qué método ("se controlarán mediante inspección")	VER SP 1.1
Armar el entorno: cargar la base de datos de pruebas	VER SP 1.2
Definir el tipo de revisión, el calendario, los roles; distribuir el producto a revisar	VER SP 2.1
Los pares chequean el producto y documentan los defectos	VER SP 2.2
Almacenar y proteger los datos de la revisión, para que no se usen para evaluar a las personas	VER SP 2.3
Controlar un producto contra sus requerimientos y registrar los resultados	VER SP 3.1
Elegir qué casos de uso entran en la prueba de aceptación	VAL SP 1.1
Acordar con el cliente los criterios de aceptación	VAL SP 1.3
Controlar con el usuario si las etiquetas de pantalla se entienden	VAL SP 2.1

16.4 Qué se valida y cómo

La validación se aplica a los productos de trabajo —requerimientos, diseños, prototipos— y al producto y sus componentes. Se hace **temprana e incrementalmente**, no al final.

El entorno debe **representar el entorno previsto**, y puede usarse completo o sólo en parte. Los métodos posibles son la discusión con usuarios, las demostraciones de prototipos, las demostraciones funcionales, los **pilotos** de materiales de formación, las pruebas realizadas por los usuarios finales y el análisis mediante simulaciones o modelado.

Es validable más de lo que se suele pensar: requerimientos y diseños, el producto y sus componentes, las **interfaces de usuario**, los **manuals de usuario**, los **materiales de formación** y la documentación del proceso.

La **verificación es incremental**: empieza por la verificación de los requerimientos, sigue con los productos de trabajo a medida que evolucionan, y culmina con la verificación del producto terminado.

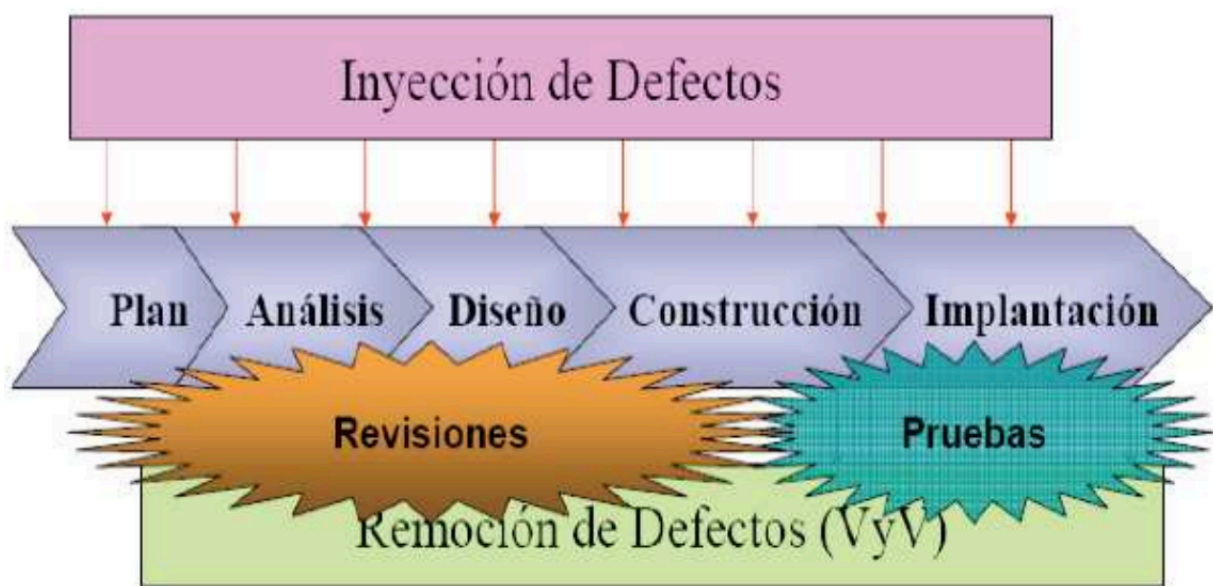
16.5 El grado de confianza

Verificar y validar no busca la ausencia total de defectos —que es inalcanzable— sino que el software sea **suficientemente bueno para su uso previsto**. Cuánta confianza hace falta depende de tres factores.

La **criticidad del sistema**: un sistema crítico exige confianza alta; un prototipo, mucho menos. Las **expectativas del usuario**, que vienen subiendo porque la tolerancia a fallos decrece. Y el **entorno de mercado**: con pocos competidores una empresa puede lanzar antes de estar completamente probada para llegar primero, y con un precio bajo los clientes toleran más defectos.

Conviene recordar que V&V son **procesos costosos**: en ciertos sistemas superan **la mitad del presupuesto total** de desarrollo. Por eso se planifican desde etapas tempranas.

16.6 Inyección y remoción de defectos



Los defectos **se inyectan en todas las fases** —plan, análisis, diseño, construcción e implantación—, no sólo al programar. Verificación y validación son el conjunto de actividades de **remoción**, y se reparten en dos franjas que se solapan: las **revisiones**, que son estáticas, cubren desde el plan hasta la construcción; las **pruebas**, que son dinámicas, cubren desde la construcción hasta la implantación.

De ahí la regla económica que ordena toda la unidad: **cuanto más tarde se detecta un defecto, más caro sale corregirlo**; una presentación de la cátedra calcula que en mantenimiento cuesta unas cien veces más que en requisitos. Por eso el riesgo de **relegar las pruebas al final** es detectar los defectos cuando corregirlos es más caro, y la respuesta es adelantar V&V: el modelo en V, la prueba en cada incremento, los prototipos y las revisiones desde los requisitos.

17. La organización de las pruebas

17.1 Qué es un caso de prueba

Caso de prueba: conjunto de **entradas, condiciones de ejecución y resultados esperados**, desarrollado para un objetivo o condición particular.

En su forma mínima se escribe como un par ordenado: (**valor de entrada** → **resultado esperado**). Sin resultado esperado no hay caso de prueba, porque no habría forma de saber si pasó o falló.

Nunca se prueba en producción: el entorno de pruebas debe estar **físicamente separado** y recrear las condiciones de producción. INTECO trata además las pruebas como un **subproyecto**, con su propio plan: en la planificación se define la estrategia, en el diseño se diseñan los casos, y en la construcción se ejecutan.

17.2 Los cuatro niveles de prueba

NIVEL	QUÉ PRUEBA	QUIÉN Y CÓMO	ÁREA
Unitarias	Cada módulo o componente aislado, antes de integrar	El propio desarrollador . Usa stubs y drivers para aislar el módulo	Las dos
Integración	La interacción entre módulos: defectos de interconexión	Se apoya sobre todo en el diseño	VER
Sistema	El comportamiento global contra la especificación, incluidos los no funcionales	Un equipo independiente , con caja negra y un entorno parecido a producción	VER
Aceptación	Que el producto satisface las necesidades del usuario	Un usuario o cliente	VAL

Ningún nivel reemplaza a otro. Que haya pruebas de integración no quita que se hagan las unitarias, y viceversa.

Para armar la integración hay tres estrategias. **Big-bang** ensambla todo de una vez: no requiere simular nada, pero consume mucho tiempo rastreando causas y descubre los problemas al final. **Bottom-up** avanza desde los módulos inferiores: necesita **drivers** en cada nivel y no encuentra problemas de diseño hasta muy avanzado. **Top-down** avanza desde los componentes superiores: necesita **stubs** que simulen los inferiores, pero **descubre rápidamente los errores de arquitectura**.

Las pruebas de aceptación **no buscan errores**: demuestran que el producto **está listo para producción**. Admiten tres modalidades: **alfa**, con un conjunto acotado de clientes preseleccionados en un entorno controlado; **beta**, con un conjunto más amplio; y **piloto**, con un conjunto reducido de departamentos del cliente y en **ambiente de producción**.

ATENCIÓN Unas pruebas que ejecuta **un equipo de la propia empresa sobre un prototipo** no son alfa ni beta, que requieren al cliente: son pruebas internas, y por área, **VER**.

17.3 Tipos de prueba

El **nivel** indica cuándo y sobre qué parte se prueba; el **tipo** indica con qué objetivo. Los tipos que se nombran en la materia son las pruebas funcionales, las de prestaciones, las de usabilidad, las de seguridad o acceso, las de configuración, las de instalación y carga inicial de datos, y las de migración de datos.

Dentro de las de **prestaciones** conviene distinguir cuatro que se confunden. Las de **carga** validan los requisitos de prestaciones definidos con escenarios realistas. Las de **capacidad** buscan el **punto umbral** a partir del cual las prestaciones se degradan, incrementando la carga hasta la saturación. Las de **estrés** estudian el comportamiento **en sobrecarga**, excediendo los límites, con foco en la integridad. Las de **estabilidad** miran el comportamiento **en el tiempo** bajo carga normal, para detectar mala liberación de recursos.

17.4 Regresión y confirmación

Son dos actividades distintas que se ejecutan juntas después de cada corrección.

La **confirmación** verifica que **el defecto corregido realmente se solucionó**: misma prueba, mismas condiciones, mismos datos. La **regresión** verifica que **el arreglo no rompió otra cosa**.

La regresión puede detectar tres situaciones: que el cambio **creó un error nuevo** (regresión local), que el cambio **reveló errores que ya existían** (de exposición), o que el cambio en un área **rompió otra área** del sistema (remota). La estrategia más simple es la **fuerza bruta**, repetir todas las pruebas, y por eso la regresión es el mejor candidato a automatizarse.

17.5 Qué es probar

Myers sostiene que las pruebas pobres nacen de una **definición falsa** de qué es probar. Hay tres que suenan razonables y son trampas de opción múltiple: que la prueba sirve para **demostrar que no hay errores**, para demostrar que el programa **funciona correctamente**, o para **establecer la confianza** en que hace lo que debe. La correcta es otra:

Prueba (Myers): el proceso de **ejecutar un programa con la intención de encontrar errores**.

De ahí salen dos ideas que van contra la intuición: un **buen caso de prueba** es el que tiene alta probabilidad de encontrar un error todavía no descubierto, y un **caso exitoso** es el que **lo descubre**. Como las pruebas exhaustivas son imposibles, la economía de la prueba pasa a ser la consideración central. Dijkstra lo resumió: "*las pruebas sólo pueden demostrar la presencia de errores, no su ausencia*".

ATENCIÓN Un sistema que pasó todas las pruebas **no está libre de errores**: sólo se puede afirmar que **pasó los casos ejecutados** y alcanzó un grado de confianza para su uso previsto.

17.6 Los diez principios de Myers

Son el marco de actitud con el que se encara el testing, y condensan buena parte de lo anterior.

1. Una parte **necesaria** de un caso de prueba es la definición de la **salida prevista**.
2. Un desarrollador debe evitar **probar su propio programa**.
3. El personal de prueba **no debería depender** del área de desarrollo.
4. **Inspeccionar los resultados** de la prueba.
5. Escribir casos **tanto para las condiciones de entrada esperadas como para las no esperadas**.
6. Examinar un programa para comprobar **que no hace lo que no se supone que haga**.
7. Evitar casos de prueba **desechables y sin documentar**: hay que guardarlos para reejecutarlos.
8. **No planificar** el esfuerzo de pruebas suponiendo que no se encontrarán errores.
9. La probabilidad de encontrar errores adicionales en una sección es **proporcional al número de errores ya encontrados** en esa misma sección.
10. Las pruebas son una tarea **altamente creativa** y un desafío intelectual.

Los principios 2 y 3 merecen desarrollo, porque suelen preguntarse. Hay tres razones por las que quien desarrolla no debe probar su propio código. La primera es de actitud: **desarrollar es un proceso creativo y probar es un proceso destructivo**, y cuesta cambiar de una mentalidad a la otra sobre el trabajo propio. La segunda es la **visión de túnel**: quien desarrolló entiende su código de raíz y tiende a probar los caminos que ya sabe que fun-

cionan. La tercera es la más grave: si el error está en **cómo se entendió el requerimiento**, quien lo entendió mal va a probar según su propio malentendido; por eso se prueba también **con el cliente**.

El principio 9 tiene una consecuencia práctica directa: conviene concentrar el esfuerzo de corrección **en los módulos donde más defectos aparecieron**, porque es donde es más probable que sigan apareciendo.

18. Las técnicas dinámicas

Las técnicas dinámicas ejecutan el código y buscan **fallos**. Existen porque **no se puede probar exhaustivamente**: las combinaciones de entradas posibles son inabarcables. Cada técnica es una forma sistemática de elegir un subconjunto de casos con alta probabilidad de encontrar defectos.

Antes de ver cada una, conviene tener el criterio de selección, porque los enunciados suelen describir la situación y pedir la técnica:

LO QUE DICE EL ENUNCIADO	TÉCNICA
Un campo con rango, longitud o formato; "valores válidos e inválidos"	Partición de equivalencia o valores límite
Varias condiciones que se combinan, reglas de negocio que se cruzan	Tablas de decisión
Una secuencia de estados, "pasa de X a Y"	Transición de estados
Flujo básico y flujos alternativos, escenarios de punta a punta	Casos de uso
Se dispone del código, cobertura de sentencias o decisiones	Caja blanca
Volver a probar lo que ya funcionaba después de un cambio	Regresión

18.1 Particionamiento de equivalencia

La idea es agrupar las condiciones de entrada que **el sistema trata igual**. Si el programa funciona para un valor de la partición, se asume que funciona para todos; si falla para uno, se asume que falla para todos. Por eso alcanza con probar **un representante por partición**.

Las particiones se dividen en **válidas** —las que el sistema debe aceptar— e **inválidas** —las que debe rechazar—. Las directrices habituales son: un rango como "10 a 100" da una partición válida —o **una por tramo**, si la regla de negocio cambia el resultado— y **dos inválidas** (por debajo y por encima); una lista como "ROJO, BLANCO, NEGRO" da una válida si todos se comportan igual, o **una por valor** si el comportamiento cambia, y una inválida; una condición de obligación como "letras mayúsculas" da una de cada una. El **vacío** va siempre como partición aparte. También existen las **particiones de salida**, que agrupan por resultado en vez de por entrada.

El formato con el que se trabaja en la materia tiene dos tablas encadenadas. La primera declara el **atributo**, su **dominio** y las particiones válidas e inválidas. Para un campo "día del mes" con dominio entero positivo entre 1 y 31, y un recargo que cambia cada diez días:

PARTICIONES	
Válidas	PV1) $1 \leq X \leq 10$ · PV2) $11 \leq X \leq 20$ · PV3) $21 \leq X \leq 31$
Inválidas	PI1) letras · PI2) $X \leq 0$ · PI3) $X > 31$ · PI4) vacío · PI5) imagen · PI6) carácter especial · PI7) cadena de caracteres

La segunda lista los casos de prueba, con un representante por partición y una fila por caso: **Caso | Partición | Entrada | Salida esperada**. Con las particiones de arriba serían diez casos: tres válidos (1, 12, 30) y siete inválidos, uno por cada forma de entrada rechazable.

ATENCIÓN Dos errores arruinan este ejercicio. El primero es **olvidar las particiones intermedias**: si un sistema se activa "por encima de 25" y se apaga "por debajo de 20", entre 20 y 25 hay una zona donde no pasa nada, y esa zona **es una partición válida**. El segundo es **dejar que las particiones se pisen**: si la válida llega hasta 31, la inválida empieza en 32, o sea $x > 31$, no $x \geq 31$.

18.2 Análisis de valor de frontera

Es una **mejora** del particionamiento, no una alternativa. En vez de un solo representante por partición, prueba **más de un caso en cada una**, concentrados en los **extremos**, porque es donde se agrupan los errores: si el código dice > 50000 en lugar de ≥ 50000 , un representante como 30.000 no lo detecta; 50.000,00 y 50.000,01 sí.

Para un rango de 10 a 100 se prueban **9, 10, 100 y 101**. Si se trabaja con dinero a dos decimales, el "siguiente valor" es 0,01. Para conjuntos ordenados, el primer y el último elemento. Hay valores especiales que conviene no olvidar: en minutos siempre 0 y 59; en fechas, los límites de mes y los **años bisiestos y no bisiestos**.

Para campos no numéricos el criterio cambia. Si el campo admite **una letra cualquiera**, alcanza un caso. Si admite un **conjunto cerrado** de N valores, va un caso por valor. Si es una **cadena de longitud fija**, se prueba con esa longitud exacta. Si acepta **hasta N caracteres**, van casos para las longitudes alrededor de N: N-1, N y N+1.

Cuando una consigna pide "cubrir todas las particiones válidas", pide **representantes**, no bordes. Cuando pide valores límite, pide los **extremos**. Es la diferencia entre las dos técnicas y es lo que se evalúa.

18.3 De las particiones a los casos de prueba

Los casos se escriben en uno de dos formatos. **Por campo**, atributo por atributo. **De función**, con una columna por atributo: se arranca con casos de **todos los valores válidos** y después va **un inválido por vez**, dejando el resto válido. En los finales se pidieron los dos formatos, así que hay que leer la consigna.

El AD de 2025 preguntó **qué hay que tener en cuenta al definir los casos de prueba a partir de las particiones**. Sí van las **entradas**, las **salidas esperadas** y el **comportamiento esperado** —es la definición misma de caso de prueba— y la **técnica a emplear**, porque partición pura y valores límite sacan valores distintos de la misma partición. No van **si la prueba es alfa o beta**, **si la ejecuta personal de desarrollo** ni las **limitaciones del lenguaje**, que clasifican pruebas pero no cambian el diseño del caso. La corrección sólo confirmó lo de la técnica y lo de alfa o beta; el resto se deduce de Myers y de INTECO.

Las resoluciones de finales que circulan como práctica repiten los mismos errores de traducción del enunciado:

EL ENUNCIADO DICE	LO CORRECTO
"Descuento si la edad es superior a 65 "	La partición empieza en 66
"Descuento si las noches son más de 6 "	Empieza en 7 ; los límites son 6 y 7
"Monto mayor a cero "	La inválida es ≤ 0 : el 0 también es inválido
"Los mayores de 17 ingresan pagando"	Partición ≥ 18 , no una partición de un solo valor
Un campo donde 0 significa "sin límite"	El 0 es válido , con partición propia
Una reunión con el cliente para evaluar la especificación	No es prueba de caja negra: es estática , y por el cliente, VAL

Y cuando el enunciado deja algo abierto, hay que **declarar el supuesto**: la cátedra corrige por la justificación.

18.4 Tablas de decisión

Se usan cuando **múltiples combinaciones de entradas** producen resultados distintos. A diferencia de las anteriores, que miran un campo por vez, esta técnica se centra en **la lógica y las reglas de negocio** que cruzan varios campos.

La tabla tiene filas de **condición** y filas de **acción**, y **cada columna es una regla de negocio**, es decir, un caso de prueba.

Armarla tiene cuatro pasos. Se identifican las **condiciones**, que son los atributos que deciden el resultado. Se identifican las **acciones**, que son los resultados distintos que el sistema puede ejecutar. Se generan las **columnas**, combinando los valores posibles de cada condición. Y se **reduce** la tabla, eliminando las combinaciones imposibles y colapsando las indiferentes.

Un ejemplo que aclara el conteo. Un gimnasio tiene socios de categoría Coulson, IronMan (10 % de descuento) o Hulk (15 %), y quien tiene la cuota al día recibe un 5 % adicional acumulable. Las condiciones son dos: la categoría y el estado de la cuota. Pero la categoría **no es binaria**: tiene tres valores mutuamente excluyentes. Entonces las columnas son $3 \times 2 = 6$, y las acciones son **cuatro**: aplicar 0 %, 10 %, 15 % y el 5 % adicional.

ATENCIÓN El error típico es contar toda condición como binaria y calcular 2^n columnas. Una condición puede tener tres o más valores, y en ese caso el producto cambia. Y a la inversa: cuando hay **condiciones incompatibles** —si el usuario no existe, la contraseña es irrelevante—, las combinaciones se colapsan y quedan menos casos de los que sugiere la potencia de dos.

18.5 Transición de estados

Se aplica a sistemas modelables como **máquina de estados finitos**, donde la salida ante la misma entrada **depende del estado anterior**. El ejemplo canónico es un trámite que pasa de Inscripto a Cursando y de ahí a Aprobado o Desaprobado.

Partiendo de la máquina de estados, la técnica permite revisar qué hace falta para llegar a cada estado y detectar incompatibilidades: **transiciones que faltan**, o estados de los que no se puede salir. Una prueba completa no se limita al camino feliz: debe incluir las **transiciones no válidas** —intentos fallidos, timeouts— y los **eventos no especificados**, como cancelar a mitad de camino.

18.6 Derivación de casos de prueba desde casos de uso

Ejercitan el sistema **de punta a punta**, siguiendo el recorrido real de un usuario. Por eso dan casos de **mejor calidad** que los armados campo por campo: validar una tarjeta en un cajero contempla muchas más cosas que probar tres números sueltos en un formulario.

El método tiene tres pasos. **Primero, los escenarios**: cada escenario es el **flujo básico** solo, o combinado con uno o más **alternativos**. En el cajero automático de la presentación de la cátedra, E1 es el retiro satisfactorio; E2, la tarjeta no válida; E3 y E4, el PIN incorrecto con y sin intentos restantes; y así con los fondos insuficientes y la cancelación.

Segundo, la matriz V/I: una fila por caso de prueba y una columna por cada condición o dato, incluidas las **condiciones que valida el sistema** aunque vengan de la base. **V** marca la condición válida, que sigue el flujo básico; **I**, la que **dispara el alternativo**; **n/a**, la que no llega a evaluarse.

CASO	ESCENARIO	TARJETA	PIN	FONDOS	RESULTADO ESPERADO
1	Retiro satisfactorio	V	V	V	Entrega el dinero
2	Tarjeta no válida	I	n/a	n/a	Expulsa la tarjeta con un mensaje
3	PIN incorrecto, sin intentos	V	I	n/a	Retiene la tarjeta
4	Fondos insuficientes	V	V	I	Mensaje; vuelve a pedir el importe

Tercero, se completa con valores reales. El primer caso es positivo del flujo básico; los demás son negativos del flujo básico y positivos de su alternativo, y un escenario puede necesitar más de un caso. Después se suman las **especificaciones complementarias**: rendimiento, seguridad, configuración e instalación.

ATENCIÓN El error típico está en la matriz. Para provocar "no hay vehículos disponibles", la resolución de un final marcaba como inválidas la fecha y la duración, que eran datos válidos: lo que dispara el alternativo es la **disponibilidad**, una condición que valida el sistema y que necesita su propia columna.

18.7 Caja negra y caja blanca

La distinción es qué información tiene quien prueba.

En **caja negra** sólo se ven entradas y salidas. Es rápida y no requiere acceso al código, pero cuando algo falla **no se sabe dónde está el error**. Todas las técnicas anteriores son de caja negra.

En **caja blanca** se dispone del **código fuente**, lo que permite saber cuántas sentencias y condiciones se ejecutaron y sacar un porcentaje de cobertura. Trabaja sobre tres estructuras: secuencia, selección e iteración.

TÉCNICA	OBJETIVO
Pruebas de sentencia	Ejecutar cada sentencia ejecutable al menos una vez
Pruebas de decisión	Evaluar cada decisión (IF-THEN-ELSE, DO-WHILE) en verdadero y falso
Pruebas de caminos	Recorrer cada camino de ejecución independiente. No prueba todas las combinaciones: con bucles serían infinitas

Conviene notar que un defecto puede manifestarse **aunque todas las sentencias se hayan ejecutado al menos una vez**, porque el problema aparece recién al **combinarse** ciertos caminos. La cobertura del 100 % de

sentencias no garantiza ausencia de defectos.

Ver el código también **mejora los casos de caja negra**. Un final lo muestra con una función que devuelve la proporción A/B entre dos enteros: con el código a la vista — `return A/B`, sin ninguna validación— aparece un caso que hay que agregar, **B = 0**. En ese momento la prueba pasó de caja negra a caja blanca.

18.8 Técnicas basadas en la experiencia

Se usan cuando **no hay una especificación adecuada** o **no hay tiempo**. La **adivinación de errores** complementa a las técnicas formales y depende de la habilidad e intuición del técnico. Las **pruebas exploratorias** consisten en recorrer el software para entender qué hace, qué no hace y dónde está débil, diseñando las pruebas mientras se ejecutan.

19. Las técnicas estáticas: revisiones

19.1 Por qué existen

Las técnicas dinámicas necesitan el sistema andando, y eso limita cuándo se pueden aplicar. Las **estáticas** analizan documentos —requisitos, diseños, código, historias de usuario— **sin ejecutar nada**, y por eso se pueden aplicar en **cualquier momento** del ciclo de vida. Buscan **defectos**, no fallos.

Son la **primera forma de prueba aplicable** en un proyecto, y están vinculadas principalmente a la **verificación**.

	ESTÁTICAS (REVISIONES)	DINÁMICAS (PRUEBAS)
Qué encuentran	Defectos	Fallos
Qué necesitan	Documentos	El sistema ejecutable
Cuándo se aplican	En cualquier momento	Sólo con el producto andando
Ventaja	Se sabe dónde está el problema; se encuentran varios a la vez; se corrige temprano	Más rápidas de ejecutar
Desventaja	Más lentas	No se sabe dónde está el error

Son **complementarias**: ninguna reemplaza a la otra.

19.2 Beneficios y costo

Las revisiones mejoran la calidad y la comprensión de los entregables, validan que soportan la solución final, gestionan las expectativas del negocio, identifican tareas de alto riesgo y forman al equipo. Al reducir los errores que llegan a la etapa de pruebas, **acortan los períodos de prueba y bajan sus costos**.

Aun así, muchas organizaciones no las implementan, y la explicación que da la materia es que tienden a **sobreestimar su costo y subestimar sus beneficios**.

19.3 Formalidad

Una revisión puede ser informal o formal, y la diferencia está en si hay proceso.

Las **informales** no tienen proceso definido, no tienen roles y habitualmente no se planean. Cualquier intercambio entre pares cuenta: preguntarle a un compañero si le parece bien un pedazo de código es una revisión informal.

Las **formales** tienen objetivos definidos, proceso documentado, roles asignados a personas entrenadas, checklists y reglas, **reporte de resultados** y recolección de datos para el control del proceso.

La formalidad importa porque deja **trazabilidad documentada** de las acciones y decisiones, lo que permite demostrar después que los procedimientos se cumplieron.

19.4 El proceso y los tipos

El proceso básico es común a todas: se identifican los entregables a revisar, se arma la lista de participantes, los revisores **estudian** el documento por su cuenta, identifican problemas y se los **comunican al autor**, y el autor **responde y actualiza**.

TIPO	QUIÉN LO DIRIGE	FOCO	FORMALIDAD
Revisión informal	—	Encontrar defectos; documentar es opcional	Mínima
Walkthrough	El propio autor	Entendimiento común, evaluar contenidos, discutir alternativas	Media
Revisión técnica	Un moderador capacitado o experto técnico	Consenso técnico , con revisores expertos; no es búsqueda de defectos	Variable
Revisión entre pares	Colegas del mismo proyecto	Identificar y eliminar defectos temprano , de forma incremental	Media
Inspección	Un moderador formado, nunca el autor	Registrar defectos eficientemente: las discusiones se posponen. Seguimiento formal con criterios de salida	Máxima

19.5 Los cinco roles y los factores de éxito

El **moderador** dirige el proceso, determina junto con el autor el tipo de revisión y la composición del equipo, y hace el seguimiento. El **autor** creó el documento y busca mejorar su calidad. El **documentador** anota cada defecto y sugerencia. El **revisor** valida el material buscando defectos **antes** de la reunión. El **supervisor** decide destinar tiempo del proyecto a las revisiones y determina si se cumplieron los objetivos.

Para que una revisión funcione hacen falta un **objetivo claro y acordado**; elegir los **documentos más críticos**, como los requisitos o la arquitectura, **sin inspeccionar todo**; reservar las horas **en el plan del proyecto**; **formar** a los participantes; crear un **entorno seguro y no amenazante, enfocado en el producto y no en la persona**; y **documentar los defectos** con ubicación y descripción.

19.6 Análisis estático

Es la variante automatizada: busca defectos **sin ejecutar** el programa, pero **una vez escrito el código**, con herramientas llamadas analizadores estáticos.

Detecta variables **no inicializadas**, variables **no utilizadas**, **código inalcanzable**, inconsistencias entre módulos, vulnerabilidades de seguridad y violaciones de los estándares de programación. Encuentra además inconsistencias en los modelos, algo que las pruebas dinámicas no pueden hacer.

Entre las métricas de código que produce, la más usada es la **complejidad ciclomática**, que se calcula como el número de sentencias de decisión binarias más uno —o, sobre el grafo del código, aristas menos nodos más dos

— y sirve para **estimar cuántas pruebas** necesita un componente.

20. La gestión de configuración (CM)

Un producto de software es un conjunto de piezas —requerimientos, diseño, código, casos de prueba, manuales y hasta las herramientas con que se construyó— que cambian todas, a ritmos distintos y en manos distintas. Cuando la software factory vende el mismo producto a varios clientes, cada uno con su versión, y el equipo corrige errores en paralelo, aparecen las frases con que la guía de la cátedra retrata a un proyecto sin gestión de configuración: "¿cuál es la versión que tiene el cliente?", "no puedo reproducir el problema en mi versión", "¿está corregido el error también en esa versión?". El riesgo principal es **entregar al cliente la versión incorrecta**, y detrás viene otro: **no poder recuperar una versión anterior** para mantener a un cliente que sigue usándola.

Gestión de configuración (CM): establecer y mantener la **integridad** de los productos de trabajo utilizando la **identificación** de configuración, el **control** de configuración, el **registro del estado** de configuración y las **auditorías** de configuración.

Integridad quiere decir saber **exactamente qué se le entregó a cada cliente** y conocer el estado y el contenido de cada versión. CM es un área **de soporte de nivel 2**: no produce nada propio, sino que pone bajo control lo que generan las demás.

20.1 Metas y prácticas

Las tres metas cuentan una historia en orden: se **crean** las líneas base, se las **mantiene** con cambios controlados, y se **documenta y audita** su integridad.

- **SG 1 Establecer líneas base.** SP 1.1 **Identificar** elementos de configuración · SP 1.2 Establecer un **sistema** de gestión de configuración · SP 1.3 Crear o liberar **líneas base**.
- **SG 2 Seguir y controlar los cambios.** SP 2.1 Seguir las **peticiones de cambio** · SP 2.2 **Controlar** los elementos de configuración.
- **SG 3 Establecer la integridad.** SP 3.1 Establecer **registros** de gestión de configuración · SP 3.2 Realizar **auditorías** de configuración.

20.2 Qué se pone bajo configuración

Elemento de configuración (EC): cualquier producto de trabajo —final o intermedio, entregable al cliente o interno— **cuyo cambio pueda resultar crítico** para el proyecto, y que se trata como **una entidad única**.

Ser EC no es una categoría aparte: un producto, un producto de trabajo interno o una herramienta pasan a ser EC cuando **se decide ponerlos bajo control**. Identificarlos (SP 1.1) es la base de todo lo demás: lo que no se identificó no se puede controlar, registrar ni auditar. Van bajo configuración los productos **usados por dos o más grupos**, los que **pueden cambiar**, los que **dependen entre sí** y los **críticos**: los planes, la especificación de requisitos y la matriz de trazabilidad, el diseño, el **código fuente**, los **casos y datos de prueba**, los manuales y todos los entregables.

ATENCIÓN Las herramientas también son elementos de configuración si hacen falta para reconstruir una versión: sin el lenguaje y el compilador **en la versión que se usó**, el código fuente no alcanza para regenerar el ejecutable.

La pregunta del año 2002. El AD de 2024 describió una software factory que desde 2002 desarrolla aplicaciones de escritorio en un lenguaje A; hace dos años sumó apps móviles en un lenguaje B; aplica un estándar de interfaces desde 2010; sus manuales eran PDF hechos con MS-Word 2002, y luego los reemplazó un módulo de manual on-line. La pregunta: "si estuviera en el año 2002, ¿qué debería quedar bajo la gestión de configuración para garantizar la evolución que se tuvo?". El criterio es **pararse en el año del enunciado** y poner todo lo que **existía entonces y hace falta para reconstruir** el software: la documentación del sistema, la app de escritorio, su **código fuente**, el **lenguaje A**, **MS-Word 2002** y los manuales en PDF. Lo que apareció después **no puede estar**: ni el lenguaje B —marcarlo dejó la pregunta en cero—, ni el estándar de interfaces, ni el módulo on-line. Queda sin confirmar si valían los manuales en .doc.

Desde la perspectiva de las **pruebas**, CM sirve para controlar la versión de los casos de prueba, **identificar qué versión del software se está probando** y seguir los cambios de los casos; no sirve para desarrollar casos nuevos.

20.3 Las líneas base

Línea base (LB): conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante el procedimiento de control de cambios**.

Para entrar en una línea base, un producto tiene que estar **acabado y formalmente aprobado**, típicamente en una **revisión formal**. Crearla es la SP 1.3, y requiere la autorización del comité de cambios. La línea base más los cambios aprobados forman la **configuración vigente**.

Lo más importante que hace una línea base no está en la definición: **relaciona entre sí las versiones de los distintos artefactos**. Así lo muestra el ejemplo de la guía para un ciclo en cascada, donde las letras son versiones:

HITO (REVISIÓN FORMAL)	CONTENIDO DE LA LÍNEA BASE
Revisión de requisitos	ERS A
Revisión de diseño	ERS B · Diseño A
Disponibilidad de las pruebas	ERS C · Diseño B · Código A · Plan de pruebas A
Aceptación	ERS D · Diseño C · Código B · Plan de pruebas B · Manual de usuario A

Sin esa relación no hay forma de saber que el producto aceptado se armó con la versión D de la especificación y la B del código. Los tipos de línea base cambian según la fuente: la guía habla de **funcional, de desarrollo y de producto**; CMMI, de **funcional, asignada y del producto**.

20.4 Versiones, variantes y releases

Con varios clientes en versiones distintas hay que poder identificar la de cada uno, saber **en qué versión entró cada cambio y reconstruirla**.

Versión: variación **temporal** de un producto o de un EC: el mismo elemento, evolucionado en el tiempo. **Variante:** variación **espacial**: el mismo producto adaptado a otro ambiente, plataforma o lenguaje, que **coexiste** con los demás.

Una app que pasa de Android 1.1 a Android 4.3 sin cambiar su funcionalidad **no es una versión nueva, es una variante**. Dentro del número de versión, la **mayor** sube con un cambio de funcionalidad, y la **menor**, con correcciones o cambios chicos.

Release: versión de lanzamiento, en la que el software **se hace público**.

De ahí sale otra pregunta del AD de 2024: el **mínimo de releases** lo decide el ciclo de vida. En **cascada** es **uno**, porque hay una sola entrega al final. En un **incremental con n incrementos** son **n**, porque cada incremento se entrega. En el enunciado real el producto se llamaba "ALFA 3" y tenía 4 incrementos: la respuesta era **4**; el "3" es parte del nombre.

La **política de versionado** es la regla escrita que dice el **formato** de la versión, **con qué valor arranca** cada componente, **cuándo sube** cada uno y si los demás **se reinician**. Si el producto está hecho de módulos, además registra qué versión de cada módulo compone cada versión del producto: eso es la línea base. No hay que confundirla con la **convención de nombres** de los archivos, que es otra cosa; la de la cátedra es **MM-TTT-AAAnn_nombre_vx_yy**, donde la versión mayor **x** vale 0 mientras el documento no es definitivo.

20.5 Cuánto sobrevive un esquema de versión

Un esquema de numeración tiene capacidad finita: con dos dígitos hay 99 valores, y cuando se agotan, el esquema "muere". El AD de 2024 pidió calcular cuántos años dura uno, y el método es este: contar los **valores** de cada componente; fijar su **frecuencia** anual; si el menor **se reinicia** cuando cambia el mayor, verificar que no se desborde dentro de un período del mayor; y si no se desborda, la supervivencia es **valores del mayor / frecuencia anual del mayor**.

El enunciado: versión **<x>.<y>**; **x** cambia con la funcionalidad, tiene dos dígitos y empieza en 01; **y** sube con cada corrección, tiene dos dígitos y **vuelve a 0 cuando cambia la funcionalidad**; la funcionalidad cambia **cada trimestre** y los errores se corrigen **cada quince días**.

¿Se desborda y?	Un trimestre tiene unas 6 correcciones quincenales: y llega a 06 o 07, lejos de 99. Manda x.
Capacidad de x:	de 01 a 99 → 99 versiones funcionales.
Tiempo:	4 cambios por año → $99 / 4 \approx 24,75$ años.
Respuesta:	24, la opción más cercana sin pasarse.

ATENCIÓN El distractor es dividir los 99 valores de **y** por las quincenas del año, como si no se reiniciara: da menos de cuatro años.

20.6 Los repositorios

No tiene sentido controlar igual el borrador que un programador tiene en su máquina y la versión entregada al cliente. Por eso el **sistema de gestión de configuración** (SP 1.2) —almacenamiento, procedimientos y herramientas— distingue tres repositorios:

REPOSITORIO	QUÉ CONTIENE	CONTROL
Dinámico (de desarrollo)	Lo que se está creando o revisando	Control de versiones , a cargo del desarrollador
Máster (controlado)	La línea base actual y sus cambios	Control de configuración completo
Estático	Las líneas base ya liberadas y archivadas	Control de configuración completo

En el día a día, un archivo no se modifica sin **desprotegerlo** (check-out) y vuelve como **versión nueva** al protegerlo (check-in). El sistema incluye también los permisos de acceso y las **copias de seguridad**, guardadas en un sitio distinto al de los originales.

ATENCIÓN **Control de versiones no es gestión de configuración formal.** Y poner bajo control los registros de **otra** área —los informes de PPQA, por ejemplo— es la práctica genérica **GP 2.6 Gestionar configuraciones** de esa área, no una práctica de CM.

20.7 El control de cambios y el comité

Los cambios son inevitables. Lo que CM garantiza es que **sólo entren los aprobados**, que se sepa **en qué versión y en qué línea base** entró cada uno, y que todos trabajen con las versiones correctas.

Comité de control de cambios (CCB): evalúa y aprueba o rechaza los cambios propuestos a los EC, asegura que se implementen los aprobados y autoriza las líneas base. En un proyecto chico puede ser el líder o una persona asignada.

Necesidad de cambio → solicitud sobre un EC → análisis de impacto → revisión del CCB
 → rechazada: se informa al solicitante
 → aprobada: se asigna, se planifica, se implementa, se prueba y se cierra

La solicitud la inicia **cualquiera, en cualquier momento**, y no trata sólo requerimientos: también **fallos y defectos**. El comité puede aceptar, modificar, rechazar o aplazar, y la decisión **siempre queda documentada**. Las dos prácticas de SG 2 se reparten el circuito: la **SP 2.1** registra la petición, analiza el impacto y la **sigue hasta su cierre**; la **SP 2.2** controla el elemento que cambia —autorización, check-in y check-out, registro del cambio y de su razón—. Las modificaciones en sí las hace el proceso técnico que corresponde; CM las registra y libera la nueva línea base.

20.8 El registro del estado y las auditorías

El **registro del estado** (SP 3.1) guarda el historial de cada EC con detalle suficiente para **recuperar versiones anteriores**, y dice **qué versión de cada EC constituye cada línea base** y qué diferencias hay entre líneas base sucesivas. Por eso "cada vez que se genera una compilación se genera un documento con las modificaciones respecto de la anterior" es SP 3.1.

Auditoría de configuración: verificación de que un EC, o el conjunto de EC que forman una línea base, se ajusta a un estándar o requerimiento especificado.

Se hace en **puntos clave** del ciclo de vida, y una auditoría exitosa es requisito para establecer la línea base del producto. CMMI distingue tres tipos: la **funcional** verifica que el EC, ya probado, logra los requerimientos de su línea base; la **física**, que el EC tal como fue construido es conforme con su documentación técnica; y la **de gestión de configuración**, que los registros son completos, consistentes y exactos.

ATENCIÓN La auditoría de configuración **no evalúa el producto en sí**: comprueba que esté **bien gestionado**, con todos los cambios registrados y trazabilidad entre cambios y productos. No es VER (¿cumple su especificación?) ni PPQA (¿se siguió el proceso de la organización?).

20.9 Cómo distinguir CM de las áreas vecinas

CM comparte vocabulario con REQM (los cambios), con PPQA (las auditorías y los estándares) y con OPD (los repositorios), y los ejercicios juegan con eso:

LO QUE DICE EL ENUNCIADO	ÁREA Y PRÁCTICA
Un conjunto de artefactos, acordados formalmente , pasa a ser la base del resto del desarrollo	CM SP 1.3
Llega un cambio y todavía se evalúa cómo afecta	REQM SP 1.3, no CM (capítulo 15)
El cambio ya se decidió aceptar y se siguen las modificaciones	CM SP 2.1
Se controla que los nombres de archivo cumplan las reglas del proyecto	CM SP 2.2
Personal externo controla que se cumpla un estándar de la organización	PPQA SP 1.2, no CM
Cada compilación genera un documento con las diferencias respecto de la anterior	CM SP 3.1
Se arma el repositorio organizacional de plantillas y procesos	OPD, no CM

El BP de versionado del AD de 2024. Una software factory vende productos a varios clientes y tiene una política que identifica la versión del producto final y la de cada artefacto, pero **cada artefacto tiene su número independiente y no se guarda la relación con las versiones de los otros**. La respuesta es que **sí** podría generar problemas de calidad, **porque si hay que volver a una versión anterior no hay información sobre la relación de versiones entre artefactos**: falta la línea base (SP 1.3) y el registro de qué versión de cada artefacto la compone (SP 3.1).

ATENCIÓN La opción "el problema principal es que no se almacenan en un **repositorio organizacional**" fue **incorrecta**: CM no exige un repositorio organizacional, eso es tema de OPD. También es falsa "es una implementación correcta de CM, dado que se conservan todas las versiones": las versiones sueltas no alcanzan.

21. El aseguramiento de la calidad (PPQA)

Un proyecto atrasado o pasado de presupuesto tiende a saltar el proceso: la revisión prevista no se hace, la trazabilidad no se actualiza, un requerimiento se cambia por teléfono. Nadie lo decide de mala fe; es lo que pasa bajo presión. PPQA existe para que alguien **que no está sometido a esa presión** mire si el proceso se sigue y se lo diga a quien puede corregirlo.

Aseguramiento de la calidad de proceso y de producto (PPQA): proporcionar al personal y a la gerencia una **visión objetiva** de los procesos y de los productos de trabajo asociados.

Es un área **de soporte de nivel 2**. Evalúa los procesos ejecutados y los productos de trabajo contra las **descripciones de proceso, los estándares y los procedimientos** aplicables; es decir, mira la **adherencia**: si se trabajó

como estaba definido. Como da servicio a todas las áreas, la práctica genérica **GP 2.9 Evaluar objetivamente la adherencia** se implementa a través de PPQA.

ATENCIÓN PPQA **no evalúa contra requerimientos**: eso es VER. PPQA asegura que **los procesos planificados se implementan**; VER, que **se satisfacen los requerimientos especificados**. Pueden mirar el mismo producto, pero desde perspectivas distintas.

21.1 Objetividad e independencia

La palabra clave del propósito es **objetiva**, y CMMI la descompone en dos ingredientes: **independencia** y **uso de criterios** que minimicen la subjetividad del revisor.

La forma tradicional de lograr independencia es un **grupo de QA independiente del proyecto**. CMMI admite otra: en una organización con **cultura abierta y orientada a la calidad** —lo más factible en organizaciones pequeñas—, la evaluación puede hacerse **entre pares**, con tres condiciones: evaluadores **formados en QA**, **separados** de quienes desarrollan el producto evaluado, y un **canal independiente** para escalar a la gerencia. Lo que excluye a alguien de evaluar un producto es **haber participado en armarlo**.

ATENCIÓN El criterio rápido para un BP: si el evaluador depende de **la misma cadena jerárquica** que el evaluado —QA dentro de Programación, o SQA y líderes de proyecto bajo el mismo gerente que decide sobre los proyectos—, **no hay independencia** y hay problema de calidad.

21.2 Metas y prácticas

- **SG 1 Evaluar objetivamente los procesos y los productos de trabajo**. SP 1.1 Evaluar objetivamente los procesos · SP 1.2 Evaluar objetivamente los **productos de trabajo** y los servicios.
- **SG 2 Proporcionar una visión objetiva**. SP 2.1 Comunicar y **asegurar la resolución de las no conformidades** · SP 2.2 Establecer **registros**.

La primera meta **evalúa**; la segunda hace que la evaluación **sirva para algo**. Personal externo que controla si se **siguió el procedimiento** definido es SP 1.1; si un **artefacto** cumple la plantilla o la directriz **de la organización**, SP 1.2; **informar, elevar o escalar** una no conformidad y seguirla hasta el cierre, SP 2.1; **registrar** las actividades de QA, SP 2.2. Si el estándar es **del proyecto**, como las reglas de nombres de archivo, no es PPQA sino CM; y usar los informes de no conformidades para **mejorar el proceso de la organización** es OPF.

21.3 Las no conformidades: del hallazgo al cierre

No conformidad (NC): problema identificado en una evaluación que refleja **falta de adherencia** a los estándares, las descripciones de proceso o los procedimientos aplicables.

La SP 2.1 es la práctica más preguntada, porque describe el **ciclo de vida de una no conformidad**: se intenta **resolverla en el proyecto**; si no se puede, se documenta y se **escala** al nivel de gerencia designado; y se **sigue cada una hasta su resolución**. Resolverla admite **tres caminos**: **corregirla, cambiar el estándar o la descripción de proceso** que se incumplió, u **obtener una excepción**.

De ahí salen afirmaciones que los parciales toman casi textualmente:

AFIRMACIÓN	RESPUESTA
"Una auditoría termina cuando se resuelven las no conformidades" (AD 2024)	Verdadera
"El trabajo de auditoría termina cuando se presentan las no conformidades" (AD 2025)	Falsa
"Las no conformidades se intentan resolver a nivel proyecto" (AD 2025)	Verdadera
"El escalamiento es un aviso que dice que no hay no conformidades" (AD 2025)	Falsa: se escala una que no se resolvió
"Las no conformidades deben escalarse aunque se hayan resuelto"	Falsa
"Una no conformidad podría cambiar la descripción de un proceso"	Verdadera

ATENCIÓN El escalamiento no puede faltar. En la corrección de un final, Ripani marcó como lo más importante de la respuesta de PPQA la frase "en caso de persistir esta situación, se podrá elevar el problema a la gerencia": **sin ella, la respuesta era incorrecta.**

21.4 El proceso de aseguramiento de la cátedra

En el proceso que adopta la cátedra, el **equipo de calidad audita periódicamente** los procesos de cada proyecto, desde el arranque hasta el cierre. El **SQA** es el asesor de calidad del proyecto: se asigna **antes de que el proyecto empiece y reporta al responsable de QA**, no al jefe de proyecto. Al planificar, colabora para que las actividades de QA queden en la **EDT y en el calendario**; durante la ejecución hace las **auditorías periódicas** e informa los hallazgos; al cierre, asegura que el informe retrospectivo llegue al histórico de la organización.

SITUACIÓN	CUÁNDO SE AUDITA
Regla general	Al final de cada fase del ciclo de vida
El paso entre dos fases se alarga mucho	Revisión bimensual
Mantenimiento evolutivo pequeño	A mitad del proyecto y antes de terminar
Pequeñas peticiones de mantenimiento correctivo	Revisión aleatoria mensual

El reparto que más sirve para los BP: **corregir la no conformidad le corresponde al jefe de proyecto**, pero **auditar, seguir y escalar le corresponde al SQA**. Se escala cuando **no se acuerda** una acción correctiva o cuando la acordada **no se cierra en fecha**, y la decisión de la autoridad superior queda registrada. El proceso y el checklist de la cátedra no nombran igual a quién se escala; lo seguro para el examen es **"fuera del proyecto, a una gerencia independiente de la que maneja el proyecto"**.

Ojo con el **sector** de SQA de una empresa: suele **evaluar** procesos y productos, que es PPQA, y también **proponer mejoras** a procesos y plantillas, que es **OPF**. "El sector de SQA comunica por mail los cambios de plantillas" describe un despliegue, OPF SP 3.1, no PPQA.

21.5 PPQA en las preguntas BP

Los dos BP de PPQA de los parciales describen la misma práctica con un final distinto: SQA controla estándares de proceso y de producto, informa las no conformidades, el líder de proyecto debe subsanarlas y SQA vuelve a revisar **a los 15 y a los 30 días**. Lo que cambia es qué pasa si la no conformidad sigue abierta:

Si la NC sigue abierta...	SQA escala al gerente de desarrollo, "con quien se tiene buena comunicación ya que tanto los PMs como SQA dependen de esa gerencia"	El informe se archiva en "proyectos incumplidores" y queda en el legajo del líder
Lo que falta	Independencia: el gerente que recibe el escalamiento manda también sobre los PMs	Escalamiento y seguimiento hasta el cierre
La opción correcta	"Sí... porque SQA debería depender de otra gerencia para evitar conflicto de intereses al escalar"	"Sí... porque no contempla un mecanismo para controlar, por un grupo externo , que se resuelvan las no conformidades hasta su fin , escalándolas si fuera necesario"

En 2024 hay escalamiento, pero va a quien es **juez y parte**, y la "buena comunicación" es un distractor. En 2025 **no hay escalamiento**: sancionar al líder no resuelve la no conformidad, y usar los datos para evaluar personas es un uso inapropiado (capítulo 22).

ATENCIÓN La opción "no contempla un mecanismo para controlar que se resuelvan las no conformidades" aparece **en los dos parciales**: en 2024 es **falsa**, porque el mecanismo existe; en 2025 es **la correcta**. Hay que leer el final del enunciado, no responder de memoria.

21.6 Los indicadores que da un checklist

El AD de 2025 preguntó qué indicadores se obtienen de un checklist aplicado a 30 casos de uso. El checklist preguntaba si se expresó la **meta**, si la sección de camino alternativo **tiene contenido** y si cada línea del historial de versiones tiene descripción y responsable. Sale "10 casos de uso sin meta", porque se cuentan los "No" de la primera pregunta. **No** sale "20 casos de uso sin caminos alternativos": en esa organización, un caso de uso sin alternativos escribe `<vacío>`, así que la sección igual tiene contenido. Y **no** sale "5 casos de uso sin historial", porque ninguna pregunta controla si el historial existe.

ATENCIÓN Un checklist sólo produce los indicadores que sus preguntas permiten contar.

22. La medición y el análisis (MA)

Las decisiones de un proyecto —¿llegamos a la fecha?, ¿el software está listo para entregar?— se toman igual, con datos o sin ellos. Medir sirve para que se tomen sobre **evidencia objetiva** y no sobre la intuición del líder. Pero medir por medir no sirve: CMMI pide que siempre haya respuesta para "**¿por qué estamos midiendo esto?**".

Medición y análisis (MA): desarrollar y sustentar una **capacidad de medición** que se utiliza para dar soporte a las **necesidades de información de la gerencia**.

Es un área **de soporte de nivel 2**, y su foco inicial es el **proyecto**: estimar con datos, seguir el rendimiento real contra el plan, detectar problemas a tiempo.

22.1 Métrica, medida e indicador

En el habla cotidiana "métrica" y "medida" son sinónimos. En la materia no. La **métrica** es la **forma de medir** un atributo, como contar líneas de código. La **medida** es el **valor** obtenido: 40.000 líneas. El **indicador** es una métrica o combinación de métricas **con criterios de decisión**, como la productividad de los programadores. Las

medidas **base** se obtienen midiendo directamente (horas, defectos); las **derivadas** se calculan a partir de otras (densidad de defectos, productividad).

Para decidir qué medir, la guía propone **GQM** (*Goal-Question-Metric*): se parte de una **meta**, se formulan **preguntas** que permitan saber si se la alcanza, y recién entonces se eligen las **métricas** que las responden. Las métricas definidas sin un objetivo explícito no sobreviven.

22.2 Primero alinear, después medir

- **SG 1 Alinear las actividades de medición y análisis.** SP 1.1 Establecer los **objetivos** de medición · SP 1.2 Especificar las **medidas** · SP 1.3 Especificar los procedimientos de **recogida y almacenamiento** · SP 1.4 Especificar los procedimientos de **análisis**.
- **SG 2 Proporcionar los resultados de la medición.** SP 2.1 **Recoger** los datos · SP 2.2 **Analizar** los datos · SP 2.3 **Almacenar** los datos y los resultados · SP 2.4 **Comunicar** los resultados.

La primera meta **define todo antes de medir**: para qué, qué, cómo, cuándo, quién y dónde. Recién la segunda **produce resultados**.

El AD de 2025 lo preguntó con cuatro afirmaciones de un equipo que va a medir plazos, defectos y productividad. Son coherentes con MA la 1, "las métricas deben estar **alineadas con los objetivos del negocio**"; la 3, "documentar **cómo y cuándo se tomarán los datos y quién los analizará**", y la 4, "el propósito de medir es obtener **evidencia objetiva** para decidir y mejorar". No lo es la 2, "**empezar a recopilar ya, sin definir los procedimientos**": saltea la primera meta. Respuesta: **sólo 1, 3 y 4**.

Usar las métricas para comparar plan contra real y corregir el proyecto **no es MA sino PMC**: MA provee la medición y PMC la usa. Estimar con datos históricos es PP. Y según un resumen de alumno, Ripani lo mostró en clase: las tres horas que un programador pasa revisando el caso de uso de un compañero son VER, y **cargarlas en el parte de horas** es MA SP 2.1.

ATENCIÓN El uso inapropiado de los datos. Al almacenar hay que prevenir que se revele información confidencial, que se interpreten datos fuera de contexto o que se **usen las medidas para evaluar a las personas**. Una "productividad por desarrollador" para premiar o castigar es señal de problema en un BP, igual que medir sin objetivo o recolectar sin procedimiento.

22.3 Métricas de proceso, de proyecto y de producto

Es lo que más se pregunta del área. La guía clasifica las métricas en tres niveles, y los parciales los preguntan de a dos.

	PROYECTO	PROCESO	PRODUCTO
Qué describen	El proyecto y su ejecución : plan contra real	Cómo rinde una actividad del ciclo de vida , para mejorar el proceso	Atributos del software
Ejemplos	Cumplimiento de plazos e hitos, esfuerzo planificado contra real, costo	Defectos detectados por fase , eficiencia de las revisiones, tiempo de corrección	Tamaño, complejidad, defectos en producción, cobertura de pruebas

La regla para decidir: si compara **lo planificado contra lo real de este proyecto**, es de **proyecto**; si mide **cómo funciona una actividad del proceso** —las revisiones, las pruebas, la detección de defectos por fase—, es de **proceso**; si mide un **atributo del software**, es de **producto**.

Con esa regla se resuelve el AD de 2024. Las métricas eran: (1) porcentaje de cumplimiento de los plazos de cada fase, (2) número de defectos identificados por cada fase del ciclo de vida, (3) horas planificadas contra reales por iteración, (4) porcentaje de tareas completadas a tiempo en cada iteración y (5) tiempo promedio de aprobación de código en revisiones. La 1, la 3 y la 4 comparan plan contra real: son de proyecto. La 2 y la 5 miden actividades del proceso. Respuesta: **proceso 2 y 5, proyecto 1, 3 y 4.**

ATENCIÓN La palabra "fase" no decide sola: la métrica 1 habla de fases y es de proyecto, porque mide el cumplimiento del plazo. Y los defectos pueden caer en cualquier nivel: "por fase" los hace de proceso, pero "defectos en producción" serían de producto.

Un indicador se documenta en una **ficha**, con su fórmula y sus **criterios de análisis**: cómo interpretar el valor y qué hacer según el resultado. Las métricas se analizan **apenas se calculan, comparándolas con los objetivos**, y para eso la guía propone las herramientas clásicas: checklist, diagrama de Pareto, histograma, gráfico de control, diagrama causa-efecto.

22.4 Dónde se guardan los datos

Los datos de medición pueden quedar en un repositorio del proyecto o, si se comparten, en el **repositorio de medición de la organización**. Ese segundo repositorio **no lo establece MA sino OPD** (SP 1.4), porque es un activo de la organización (capítulo 7). MA, de nivel 2, guarda los datos que produce; OPD, de nivel 3, crea el repositorio común. Y las prácticas de MA son **las mismas** en cualquier nivel: lo que aparece en el nivel 3 es ese repositorio.

23. Las pericias informáticas

La informática aparece hoy en casi cualquier conflicto judicial, como herramienta del delito o como su objeto. Y plantea un problema que otras pruebas no tienen en la misma medida: la evidencia digital **se altera con facilidad**. Encender un equipo apagado puede modificar fechas, y ejecutar cualquier programa puede cambiar las fechas de acceso de los archivos. La disciplina existe para que la prueba llegue al juez **indubitable**.

23.1 Forensia, pericia y perito

Informática forense: la ciencia de adquirir, preservar, obtener y presentar datos que han sido procesados electrónicamente y guardados en un medio computacional.

La **pericia informática** es el **acto procesal**: se pide cuando, para descubrir o valorar una evidencia, hacen falta conocimientos especiales. La forensia es la herramienta; la pericia, el acto en que se usa. El **perito** trabaja **en el laboratorio** como asesor científico del juez o el fiscal; el allanamiento y el secuestro **no son tareas suyas**, sino de la **policía con el fiscal**. Conviene que los **puntos de pericia** sean precisos: si el juez pide "buscar evidencia" sin palabras clave, el resultado queda librado al criterio del perito.

23.2 El principio de Locard y las fases

Principio de intercambio de Locard: siempre que dos objetos entran en contacto, transfieren parte del material que incorporan al otro.

Todo delito deja rastro, y **el propio análisis también lo deja**: de ahí la regla que ordena la unidad, **afectar el sistema lo menos posible**. El procedimiento tiene que ser **verificable, reproducible, documentado e independiente** de quién lo haga. Recorre cinco fases: **preservación** (que no se pierdan evidencias), **adquisición** (recolectarlas, con sus hashes), **análisis, documentación** (fotografías, cadena de custodia, bitácora y dos informes, uno ejecutivo y uno técnico) y **presentación** (conclusiones objetivas, sin afirmaciones no demostrables).

ATENCIÓN Las fases **no son estrictamente secuenciales**: están entrelazadas. La documentación empieza en la preservación, no cuando termina el análisis.

23.3 La evidencia y el orden de volatilidad

Evidencia: cualquier prueba que pueda usarse en un proceso legal.

Para servir tiene que ser **admisible** (cumple la legislación), **auténtica** (sin manipulación), **completa, confiable** (sin dudas sobre cómo se obtuvo) y **creíble** (comprensible para un tribunal).

La otra distinción es la **volatilidad**. La memoria, las conexiones de red y los procesos **se pierden al apagar**; el disco y los logs sobreviven. El RFC 3227 manda recolectar **de mayor a menor volatilidad**: registros y caché; tablas de enrutamiento, procesos y memoria; información temporal; disco; logs; topología de la red; y por último documentos. Y prohíbe **apagar** antes de tener lo volátil y **confiar en los programas del sistema investigado**: se usan herramientas propias, desde un medio de sólo lectura.

ATENCIÓN La **memoria va antes que el disco**: tomar primero la imagen del disco, o apagar el equipo para trabajar tranquilo, contradice el orden.

23.4 La adquisición: imagen forense, hash y bloqueo de escritura

La regla es **no trabajar nunca sobre el original**.

Imagen forense: copia **bit a bit** de la evidencia, que se hace **sólo para analizar sobre ella** y que conserva toda la información, incluida la oculta o remanente.

No hay que confundirla con un **backup**, que es una **copia simple de archivos**: es invasivo, altera la evidencia y no conserva la información oculta o remanente. **El perito no hace backups**: el protocolo los excluye de las tareas periciales.

Para probar que la imagen **no se modificó después**, se calcula su **hash** y se anota en la cadena de custodia: si al recalcularlo coincide, la imagen es la misma. **MD5 tiene colisiones** —dos archivos distintos pueden dar el mismo valor— y conviene **SHA-256 o SHA-512**. Al copiar se usa un **bloqueador de escritura**, por hardware o por software, que impide escribir sobre el original. Su límite se pregunta: un **disco de estado sólido con TRIM** borra las celdas liberadas **con sólo tener corriente**, y el bloqueador no lo evita.

23.5 La cadena de custodia

Entre el secuestro y el perito pasan **mucho tiempo y muchas manos**: allanamiento, comisaría, juzgado, laboratorio. Sin registro no se puede probar que la evidencia no se manipuló.

Cadena de custodia: registro documentado de **dónde, cuándo y quién** descubrió, recolectó, manejó y custodió la evidencia, y de **cada cambio de manos**.

Empieza en el **primer contacto** con la evidencia —normalmente el secuestro—, no cuando llega al perito, y no es una simple formalidad: es la historia clínica de la evidencia. En el protocolo judicial, la integridad física se asegura con **precintos** que la policía coloca desde el secuestro, y sus números de serie van en el acta y en el oficio.

ATENCIÓN Si un precinto llega **roto** y no hay documentada una intervención forense calificada, **no se puede asegurar la integridad y se pierde ese medio de prueba**.

23.6 El protocolo de actuación

El protocolo de actuación para pericias informáticas del Poder Judicial de Neuquén busca **evitar la contaminación de la prueba** y **formalizar** el procedimiento. Lo divide en dos etapas: la **incautación y la cadena de custodia**, a cargo de la policía con el fiscal, en el lugar del hecho; y el **análisis y el informe pericial**, a cargo del perito, en el laboratorio. Por regla se prefiere el **secuestro** a peritar en el lugar, el material llega siempre con el **oficio que fija los puntos de pericia**, y quedan excluidas las tareas ajenas a la disciplina, como transcribir, imprimir o hacer backups.

Lo que más se confunde es qué hacer con un **equipo encendido**, porque las fuentes no dicen lo mismo. **INCIBE**, que le habla a un técnico que responde a un incidente, manda **no apagarlo** hasta haber recolectado la información volátil. El **protocolo de Neuquén**, que le habla a un policía en un allanamiento, manda dejarlo prendido y **consultar a un especialista**; sin asesoramiento, **desenchufarlo del lado del gabinete**, nunca de la pared. Hay que identificar qué fuente cita la pregunta.

24. Cómo reconocer el área de proceso en un enunciado

El ejercicio más repetido de la materia es una variación de la misma consigna: "*relacione la actividad descripta con el área de proceso y la práctica específica que corresponda, o indique NINGUNA*". Parece de memoria y no lo es. Casi cualquier actividad se puede forzar en dos o tres áreas, y lo que decide es leer bien **quién hace qué, sobre qué objeto, contra qué** lo compara y **en qué momento** ocurre. Este capítulo junta los criterios de los capítulos anteriores en un solo lugar, para consultarlo durante el examen.

24.1 El método en cinco preguntas

1. **¿Es de un proyecto o de toda la organización?** Si dice "en el proyecto X", la respuesta está entre PP, PMC, REQM, RD, CM, VER, VAL, RSKM y PPQA. Si dice "en la software factory" o "para todos los proyectos", lo más probable es OPD, OPE, OT, o MA cuando define cómo se mide en toda la organización.
2. **¿Cuál es el objeto?** Un requerimiento, un elemento de configuración, un plan, un producto de trabajo, un activo, la formación de la gente. El objeto descarta áreas enteras: un activo nunca lo controla VER.
3. **¿Contra qué se compara y quién participa?** Contra la especificación o los artefactos del proyecto, por el equipo → **VER**. Con el cliente, contra lo que necesita → **VAL**. Contra un estándar de la organización, por alguien externo al proyecto → **PPQA**. Contra una convención del propio proyecto → **CM**. Contra el plan → **PMC**.
4. **¿En qué momento está el relato?** El tiempo verbal elige la práctica dentro del área y separa áreas vecinas (24.2).

5. **¿La opción tiene la sigla correcta?** Una práctica real con la sigla de otra área es falsa, y si ninguna opción encaja, la respuesta es **NINGUNA**.

Un ejemplo de punta a punta. "Ayer, dos analistas controlaron los casos de uso de otro analista contra las minutas aprobadas; los hallazgos ya se documentaron; en este momento se almacenan las identidades de los participantes donde sólo accede Calidad." Es un proyecto; el objeto es un producto de trabajo; se compara contra artefactos del proyecto, entre pares: **VER**. Y como la revisión ya ocurrió, se **protegen los datos: VER SP 2.3**. La mención de Calidad es un distractor.

24.2 El tiempo decide la práctica

LO QUE DICE EL ENUNCIADO	LO QUE CORRESPONDE
"Se distribuirá", "será chequeada" (futuro)	Preparar: SG 1 de VER o de VAL, o VER SP 2.1 si es una revisión entre pares
"Está chequeando", "se realiza un control" (presente)	Realizar o llevar a cabo
"Ya se revisó y se documentó; ahora se almacena..."	Analizar los datos: VER SP 2.3
"Previo al inicio"; reunión de lanzamiento futura	PP
"En la última reunión de avance", "respecto a lo planificado"	PMC
"Se concluyó que la causa es..." / "se determinó reemplazar..." / "ahora se evalúa si funcionó"	PMC SP 2.1 / 2.2 / 2.3
Un cambio cuyo impacto todavía se evalúa / uno ya aceptado en trámite	REQM SP 1.3 / CM SP 2.1
Una mejora propuesta , a probar en pilotos / una ya aceptada , a desplegar	OPF SG 2 / SG 3

24.3 Los pares que se confunden

PAR	CÓMO SE CORTA
PPQA / VER	PPQA controla que el producto respete el estándar de la organización; VER, que su contenido sea correcto contra lo especificado
PPQA / VAL	Un checklist del estándar aplicado antes de entregar , sin el cliente, es PPQA SP 1.2; con el cliente, VAL
PPQA / CM	Estándar de la organización , controlado por alguien externo → PPQA; convención del propio proyecto → CM
PPQA / PMC	Comparar cómo se ejecutaron las tareas contra el proceso definido → PPQA SP 1.1; contra el plan → PMC
PPQA / OPF	PPQA evalúa e informa ; OPF analiza esos informes y propone la mejora
OPF / OPD	OPF propone, pilotea y despliega ; OPD crea y deja asentado el activo
OT / PP SP 2.5	Capacidades para la organización → OT; cursos para la gente de un proyecto → PP SP 2.5
PP SP 1.2 / SP 1.4	Tamaño → SP 1.2; esfuerzo o costo derivados del tamaño → SP 1.4
PP SP 2.3 / MA	Qué datos se registran en un proyecto → PP SP 2.3; cómo se mide en toda la organización → MA
MA / OPD	MA especifica medidas y procedimientos; la estructura del repositorio de la organización es OPD SP 1.4

PAR	CÓMO SE CORTA
MA / PMC	MA produce la medición; usarla para corregir el proyecto es PMC
REQM / RD	RD produce requisitos todavía no acordados; REQM gestiona los acordados
RD SP 3.5 / VAL	Validar los requisitos antes de construir → RD; el producto con el cliente → VAL
VER / OPD	Pares que revisan un producto de un proyecto → VER; un activo aún no publicado → OPD
PP / PMC / RSKM	Riesgos para el plan → PP SP 2.2; seguirlos → PMC SP 1.3; fuentes, parámetros, estrategia y planes de mitigación → RSKM

24.4 De la palabra del enunciado al área

La tabla junta las expresiones que se repiten en los finales y en la práctica del AD. No reemplaza al método: sirve para confirmar una respuesta o para destrabarse cuando dos áreas parecen posibles.

LO QUE APARECE EN EL ENUNCIADO	ÁREA / PRÁCTICA
Una plantilla o un procedimiento para toda la SF	OPD SP 1.1
Las fases " desde que nace la idea hasta que el software es retirado "; aprobar los ciclos de vida	OPD SP 1.2
" En base a qué características " se elige un ciclo de vida o una plantilla; pedir una excepción	OPD SP 1.3
La base de datos de métricas de todos los proyectos	OPD SP 1.4
El sitio de Intranet con los documentos del proceso	OPD SP 1.5
Herramienta o software estándar para todas las PC de la SF	OPD SP 1.6
Comparar los procesos con CMMI; puntos fuertes y débiles	OPF SP 1.2
Qué proyectos piloto hacen falta para verificar una propuesta	OPF SP 2.2
Qué proyectos en curso deben incorporar la reciente modificación	OPF SP 3.2
Incorporar lecciones aprendidas a los activos	OPF SP 3.4
Conseguir un instructor ; formar a alguien para que capacite a otros	OT SP 1.4
Guía para todos los proyectos de cómo y cuándo registrar las horas	MA SP 1.3
Calcular los puntos función / las horas a partir del tamaño	PP SP 1.2 / SP 1.4
Cantidad de personas y dedicación del equipo	PP SP 2.4
Tabla de roles con la dedicación de los interesados por fase	PP SP 2.6
Si los usuarios clave asisten a las reuniones o responden los mails	PMC SP 1.5
Informar el aumento de probabilidad de un riesgo respecto de lo planificado	PMC SP 1.3
" Toda solicitud que no cumpla será devuelta "	REQM SP 1.1
Acordar con el Sponsor que la fecha se corre por un cambio	REQM SP 1.2
Matriz de trazabilidad	REQM SP 1.4
Un requisito vago que hay que volver medible y confirmar con el cliente	RD

LO QUE APARECE EN EL ENUNCIADO	ÁREA / PRÁCTICA
Códigos de los casos de uso; responsable de cada artefacto	CM SP 1.1
Procedimiento de back up ; carpetas y permisos del repositorio	CM SP 1.2
Artefactos revisados y acordados como " basamento " del desarrollo	CM SP 1.3
Documento con las diferencias entre una compilación y la anterior	CM SP 3.1
Personal externo controla un artefacto contra las directrices de la SF	PPQA SP 1.2
Elevar un informe al Director tras observaciones reiteradas	PPQA SP 2.1
Qué clases entran en la prueba de integración ; controlar por inspección	VER SP 1.1
Cargar la base de datos de pruebas	VER SP 1.2
Distribuir un caso de uso que será chequeado ; definir que será un walkthrough	VER SP 2.1
Qué casos de uso entran en la prueba de aceptación	VAL SP 1.1
Los criterios con los que el cliente da por cumplido el contrato	VAL SP 1.3

24.5 Trampas que ya se tomaron

Leer la sigla, no sólo el texto. Si la respuesta es VER y la opción dice "VAL / SP 2.1 Preparar las revisiones entre pares", la respuesta es **NINGUNA**.

El área la define la actividad, no el tema. Elevar al Gerente General un informe porque se cambian requerimientos sin seguir el circuito de REQM es **PPQA SP 2.1**: lo que se hace es escalar una no conformidad. Un enunciado que menciona la línea base de requerimientos no es por eso REQM: si se avisa que aumentó la probabilidad de un problema, es riesgo (PMC SP 1.3).

Nombrar a Calidad no lo vuelve PPQA. PPQA aparece cuando se controla la **adherencia a un estándar o a un proceso** de la organización, no cuando aparece la palabra "Calidad".

Las pruebas no se reparten por fase. La aceptación es VAL y la de sistema es VER, aunque las dos se hagan sobre el producto terminado.

RSKM casi nunca es la respuesta. En los finales no aparece, y en la práctica oficial del AD ni siquiera figuraba entre las opciones. Sólo va cuando el enunciado habla de fuentes, parámetros, umbrales, estrategia o planes de mitigación.

En algunos enunciados las resoluciones de alumnos dan prácticas distintas, pero casi siempre el **área** no se discute. Si dos prácticas parecen posibles, elegí la que aparezca entre las opciones.

25. Las preguntas BP

En los AD de 2024 y 2025, el formato que más pesó no pide una definición: describe cómo trabaja una software factory —una política, un procedimiento, una plantilla— y pide decidir si esa práctica **puede generar problemas de calidad** y **por qué**. Lo que se evalúa es si se entiende **para qué sirve** cada práctica de CMMI: se muestra una práctica a la que le falta algo, y hay que darse cuenta de qué falta y qué consecuencia tiene.

25.1 El método

1. **Leer la práctica como un proceso:** quién hace qué, contra qué criterio, cuándo y, sobre todo, **qué pasa al final**. El problema casi siempre está en la última oración: el escalamiento, el archivo, el registro, quién aprueba.
2. **Identificar el área de proceso** con los criterios del capítulo 24.
3. **Buscar el eslabón que falta** (25.3). Si falta uno, **sí** hay problema. "No genera problemas" es correcto sólo si la práctica cumple lo que pide el área.
4. **Evaluar cada razón por separado.** Un "Sí... porque X" es correcto sólo si X es **cierto, ataca el eslabón que falta** y pertenece **al área en juego**.
5. **Aplicar la regla de no mezclar** y recién ahí marcar.

25.2 La regla de no mezclar

La consigna lo dice textualmente: pueden elegirse varias opciones, pero **no pueden combinarse** las que consideran que sí hay problemas de calidad con las que consideran que no. Si se marcan varias, tienen que ser todas del mismo lado y todas con razones verdaderas. Y si hay un problema pero ninguna de las razones ofrecidas lo describe, la respuesta es **NINGUNA**, no la menos mala.

ATENCIÓN Un "Sí... porque X" con un X falso es tan incorrecto como un "No". Buena parte de los distractores dicen "Sí" y fallan en la razón.

25.3 Señales de problema por área

ÁREA	SEÑALES DE PROBLEMA	LO QUE FALTA
PPQA	Evaluator bajo la misma gerencia que decide sobre el proyecto · no conformidades que nadie sigue hasta el cierre · sin escalamiento · la no conformidad se archiva o se sanciona	Independencia · escalar y seguir hasta la resolución (SP 2.1)
CM	Artefactos versionados sin relación entre sí · sólo se guarda la última versión · documentos en carpetas personales · cambios sin petición ni autorización · pase a producción verbal	Líneas base (SP 1.3) · control de cambios (SG 2) · registros (SP 3.1)
PP	Estimar sólo por experiencia · tareas sin EDT · no se planifican los roles del cliente · riesgos sin identificar	EDT · estimaciones con datos (SP 1.2, 1.4) · involucración (SP 2.6)
PMC	Reuniones de avance sin el cliente · desvíos que no se analizan	Monitorizar contra el plan · acciones correctivas
REQM	Cambio pedido por mail al programador , que lo acepta solo · sin análisis de impacto · sin trazabilidad	Gestionar los cambios (SP 1.3) · trazabilidad (SP 1.4)
RD	Requisitos verbales · el cliente no ve nada hasta la instalación · relevamiento sin técnicos	Desarrollar y validar los requerimientos
MA	Se mide sin objetivo · factor de productividad de mercado nunca recalibrado · métricas para evaluar personas	Objetivos (SP 1.1) · uso apropiado de los datos
VER	El programador prueba su propio programa · datos de prueba decididos al ejecutar · casos sin resultado esperado · en mantenimiento se prueba sólo lo modificado	Procedimientos y criterios · revisiones entre pares · registro
VAL	La única prueba es la del usuario · sin criterios de aceptación acordados	Preparar y realizar la validación

ÁREA	SEÑALES DE PROBLEMA	LO QUE FALTA
OPF	Mejoras comunicadas por mail y nada más · lecciones aprendidas archivadas sin que nadie las consulte	Desplegar · monitorizar la implementación · incorporar experiencias
OPD	Cada grupo usa su propia estructura de documento · sin guías de adaptación	Procesos estándar · guías de adaptación
OT	Personal sin formación en las técnicas que usa la SF · no se mide si la capacitación sirvió	Plan de formación · registros · eficacia
RSKM	Los riesgos se tratan cuando ya ocurrieron	Preparar, identificar y mitigar antes

Una práctica puede fallar **por defecto** —no se mide nada— o **por exceso o mala aplicación**: se mide todo sin objetivo, o un proyecto chico completa la plantilla de uno grande porque no hay guías de adaptación. Las dos formas generan problemas de calidad.

25.4 Lo que no es un problema

Hay rasgos que parecen sospechosos y son exactamente lo que pide el modelo. Que haya **testers dedicados exclusivamente** a probar es la independencia que piden los principios de Myers. Que la no conformidad se intente resolver **primero dentro del proyecto** es lo que pide CMMI. Que el **líder de proyecto sea responsable de corregir** es cierto; lo que no puede faltar es que QA siga la no conformidad y la escale. Un QA **hecho por pares** en una organización chica es válido si se cumplen las condiciones del capítulo 21. Y que el repositorio sea **por proyecto** no es lo que exige CM.

25.5 Los distractores que se repiten

PATRÓN DEL DISTRACTOR	POR QUÉ ES FALSO
"No genera problemas: es una implementación correcta (o completa) de X"	Justamente le falta un eslabón de X; y a veces X ni siquiera es el área en juego
Una razón de otra área : "SQA no verifica que se cumplan los requisitos" (VER), "no se guarda en un repositorio organizacional" (OPD)	La razón tiene que atacar la falla del área que se está implementando
Una razón inventada : "faltan almacenar métricas de las no conformidades escaladas al área de testing"	No es lo que pide el modelo ni lo que falla en el enunciado
La sanción como solución: "si no las resuelve, es sancionado en su legajo"	Sancionar no resuelve la no conformidad, y usar datos para evaluar personas es un uso inapropiado
La buena comunicación en lugar del mecanismo	No reemplaza la independencia ni el registro
" No es necesario definirlo en cada proyecto: está en los manuales de la empresa"	CMMI lo pide por proyecto

Los tres casos de los AD están resueltos en sus capítulos: el plan de proyecto sin los roles del cliente (10.2), el versionado sin línea base (20.9) y SQA en sus dos versiones (21.5). Un cuarto, de un final, completa el cuadro: una software factory con **testers dedicados** que deciden los datos de prueba al ejecutar **sí** tiene un problema, pero **por los datos, no por los testers**: faltan casos documentados antes de ejecutar, que además permitan repetir las pruebas en mantenimiento. Una opción que dijera "Sí, porque los testers no deberían dedicarse sólo a probar" sería falsa aunque el "Sí" fuera correcto.

ATENCIÓN En una recomendación de PPQA, el **escalamiento** no es opcional: es la idea que decidió los BP de SQA del AD de 2024 y de 2025.

26. Confusiones frecuentes

Para cerrar, las confusiones que más se repiten. No son trucos de examen: cada una es un concepto mal entendido, y casi todas aparecieron como opción incorrecta en algún parcial o final.

Sobre la calidad y CMMI. Creer que asegurar la calidad es probar el producto, cuando las pruebas son **control** y el aseguramiento es **preventivo** y mira el **proceso**. Confundir el mantenimiento **perfectivo**, que mejora la calidad interna, con el **evolutivo**, que cambia lo que el sistema hace. Creer que las prácticas son obligatorias, cuando lo requerido son **sólo las metas**. Creer que un nivel superior exige de las metas de los anteriores, cuando los niveles son **acumulativos**: si falla un área de nivel 2, la organización queda en **nivel 1**. Y creerle a las erratas de la traducción: OT y RSKM son de **nivel 3**.

Sobre la gestión de procesos. Confundir OPF con OPD: **OPF diagnostica y despliega, OPD define y guarda**. Atribuir a MA el repositorio de medidas de la organización, que es de **OPD**. Llamar OT a la capacitación de la gente de un proyecto, que es **PP SP 2.5**. Y confundir la plantilla de caso de uso, que es un **activo**, con el caso de uso, que es un **producto de trabajo**.

Sobre la planificación y los riesgos. Sumar **duración** donde se pide **esfuerzo**: el esfuerzo se suma; la duración sale del camino crítico. Confundir **tamaño** (PP SP 1.2) con **esfuerzo** (PP SP 1.4). Creer que cualquier desvío debe resolverse, cuando PMC actúa sobre los **significativos** y RSKM, al superar un **umbral**. Creer que el análisis de riesgos va antes que la identificación. Y confundir **mitigación**, que actúa antes, con **contingencia**, que responde cuando el riesgo ya ocurrió.

Sobre los ciclos de vida. Creer que en un incremental el análisis y la prueba de sistema se hacen sólo una vez, cuando **cada incremento es una cascada completa**. Confundir incremental, que agrega pedazos distintos, con iterativo, que **vuelve sobre los mismos** requisitos. Y elegir cascada con requisitos poco claros, cuando es **el menos adecuado**.

Sobre los requerimientos. Creer que los no funcionales son opcionales. Elegir la opción en la que el equipo **decide solo**, cuando RD exige confirmar con el cliente. Elegir la que **implementa antes de evaluar el impacto**. Separar REQM de CM por quién pide el cambio, cuando decide si el cambio **altera un requisito** y en qué **momento** está. Y atribuir las solicitudes de cambio a REQM, cuando son de **CM** y cubren también defectos.

Sobre verificación, validación y pruebas. Creer que las pruebas de sistema son validación porque se hacen sobre el producto terminado, cuando son **verificación**. Atribuir a VAL las revisiones entre pares, que existen **sólo en VER**. Creer que probar es demostrar que el programa funciona, cuando es **buscar errores**. Creer que re-ejecutar los casos que fallaron alcanza, cuando eso es **confirmación** y falta la **regresión**. Y leer mal el enunciado: "superior a 65" empieza en **66**, y la inválida de "mayor a cero" es ≤ 0 .

Sobre la gestión de configuración. Creer que conservar todas las versiones de cada artefacto es hacer gestión de configuración, cuando sin **línea base** no se sabe qué versión de cada uno compone el producto. Dejar afuera las **herramientas** con que se construyó el producto. Contar como release cada versión interna, cuando release es **lo que se hace público**. Llamar versión a una adaptación a otra plataforma, que es una **variante**. Y creer que CM exige un repositorio organizacional, que es de **OPD**.

Sobre el aseguramiento de la calidad. Creer que la auditoría termina con el informe, cuando termina cuando las no conformidades **se resuelven**. Creer que el escalamiento es opcional, o que archivar el informe o sancio-

nar al líder lo reemplaza. Creer que alcanza con escalar a un gerente "con quien hay buena comunicación", cuando si también manda sobre los proyectos es **juez y parte**. Y creer que PPQA verifica requerimientos, cuando evalúa la **adherencia a procesos y estándares**.

Sobre la medición. Creer que toda métrica que menciona fases es de proceso, cuando el cumplimiento de plazos compara plan contra real y es de **proyecto**. Creer que conviene empezar a medir cuanto antes, cuando primero se **alinea** con los objetivos. Y creer que medir la productividad de cada desarrollador para premiarlo o sancionarlo es buena práctica.

Sobre las pericias. Creer que el perito hace un backup, cuando hace una **imagen forense bit a bit**. Tomar el disco antes que la memoria, cuando el orden va de **mayor a menor volatilidad**. Y creer que la cadena de custodia empieza cuando el material llega al perito, cuando arranca en el **primer contacto** con la evidencia.